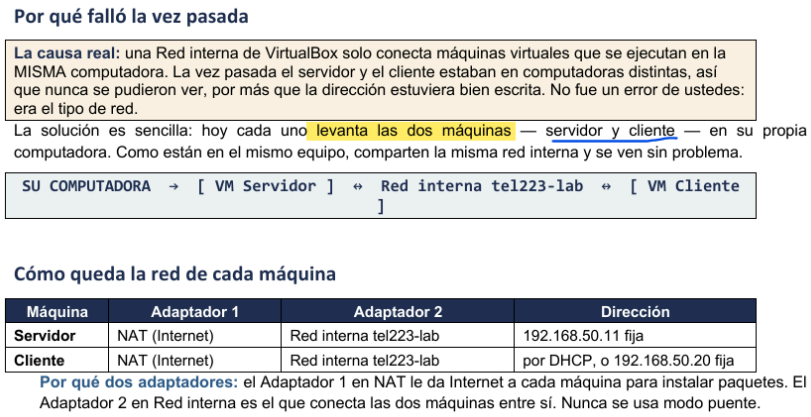
LABORATORIO #3 - RED CON DOS MÁQUINAS Y SERVIDOR WEB

ETAPA 1



Por qué falló la vez pasada

La causa real: una Red interna de VirtualBox solo conecta máquinas virtuales que se ejecutan en la MISMA computadora. La vez pasada el servidor y el cliente estaban en computadoras distintas, así que nunca se pudieron ver, por más que la dirección estuviera bien escrita. No fue un error de ustedes: era el tipo de red.

La solución es sencilla: hoy cada uno levanta las dos máquinas — servidor y cliente — en su propia computadora. Como están en el mismo equipo, comparten la misma red interna y se ven sin problema.

```
SU COMPUTADORA → [ VM Servidor ] ↔ Red interna tel223-lab ↔ [ VM Cliente ]
```

Cómo queda la red de cada máquina

Máquina	Adaptador 1	Adaptador 2	Dirección
Servidor	NAT (Internet)	Red interna tel223-lab	192.168.50.11 fija
Cliente	NAT (Internet)	Red interna tel223-lab	por DHCP, o 192.168.50.20 fija

Por qué dos adaptadores: el Adaptador 1 en NAT le da Internet a cada máquina para instalar paquetes. El Adaptador 2 en Red interna es el que conecta las dos máquinas entre sí. Nunca se usa modo puente.

▼ ¿Por qué existen dos adaptadores, qué son?



Para explicar la tabla de la imagen **"image_66b180.png"** y la guía **"Guia Laboratorio3 Red ServidorWeb TEL223.pdf"** a alguien sin conocimientos previos, podemos imaginar que una computadora es como una casa y las máquinas virtuales son habitaciones dentro de esa casa.

Aquí tienes la explicación paso a paso de toda la lógica:

¿Qué es un adaptador y de dónde viene?

- En el mundo real, un adaptador de red es la pieza física de tu computadora (como el enchufe para el cable de internet o la antena Wi-Fi) que te ***permite comunicarte*** con otras computadoras.

- En el caso de las "máquinas virtuales" (computadoras simuladas por un programa de software), estas piezas físicas no existen. El programa (como VirtualBox) "***inventa***" o ***simula*** estas piezas. Por lo tanto, el adaptador "viene" del software mismo, que crea una puerta virtual para que esa máquina imaginaria pueda comunicarse con el exterior o con otras máquinas.

¿Por qué cada máquina virtual tiene 2 adaptadores y cuál es la lógica?

- Imagina que la máquina virtual es una habitación que necesita dos puertas distintas porque tienen propósitos completamente separados que no deben mezclarse.
- Una puerta (Adaptador 1) se usa exclusivamente para ***salir*** a la ***calle*** y conseguir provisiones, mientras que la otra puerta (Adaptador 2) da a un pasillo ***privado*** que solo conecta con la habitación de al lado.

¿Cuál es la lógica del NAT, qué es y por qué dice (Internet)?

- NAT (Traducción de Direcciones de Red) actúa como un recepcionista o traductor. La máquina virtual no tiene un contrato de internet propio. El ***modo NAT*** hace que la máquina virtual le ***pida prestado el internet a tu computadora*** real de forma transparente. ***Agarrando la IP de la computadora***.
- La tabla dice explícitamente "(Internet)" porque el único trabajo de este Adaptador 1 es darle salida a Internet a cada máquina para que puedan descargar e instalar paquetes o programas necesarios para funcionar.

¿Cuál es la lógica de que el adaptador 2 sea la "red interna"?

- Una "red interna" es como un pasillo cerrado que no tiene salida a la calle.
- La lógica de usar este modo en el Adaptador 2 es crear una conexión privada ("tel223-lab") exclusiva para que la máquina Servidor y la máquina Cliente se comuniquen entre sí, aisladas de

cualquier otra red externa o del laboratorio físico. ***Las redes internas solo conectan máquinas que viven en la misma computadora real***.

¿Cuál es la lógica de que un adaptador sea un protocolo (NAT) y el otro un tipo de red (Red interna)?

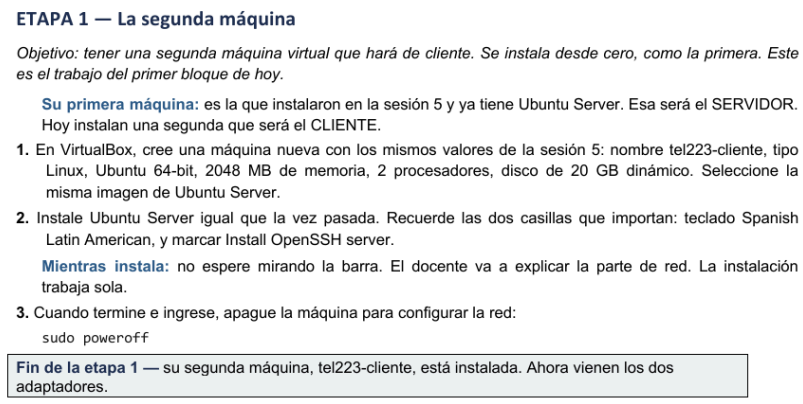
- Esta es una pequeña confusión común. En este contexto, "NAT" y "Red interna" son simplemente los nombres que el programa VirtualBox le da a sus "***modos de conexión***".
- VirtualBox te pregunta: "¿Cómo quieres que funcione esta puerta?". Para el primer adaptador, eliges la técnica de compartir internet llamada "NAT". Para el segundo adaptador, eliges el modo de aislamiento llamado "Red interna". No es que se estén mezclando peras con manzanas; simplemente se están eligiendo dos herramientas de configuración diferentes que el software ofrece para resolver dos problemas distintos.

¿Cuál es la lógica de las direcciones?

- Si los adaptadores son las puertas, las direcciones (direcciones IP) son los números exactos pintados en esas puertas para que el correo sepa dónde entregar un mensaje.
- ***Servidor (192.168.50.11 fija)***: ***Se le asigna una dirección "fija"*** (que nunca cambia) porque un servidor es como un negocio que ofrece un servicio. Si el negocio se mudara de dirección todos los días, los clientes no sabrían dónde encontrarlo. Por eso, el servidor siempre está en la `.11`.
- ***Cliente (por DHCP, o 192.168.50.20 fija)***: Se le da la dirección `.20` para que también tenga un número de puerta en el pasillo privado de la red interna. "Por DHCP" simplemente significa que el número se le podría asignar de forma automática si fuera necesario.

💡 ¿Por qué me interesaría saber que en este laboratorio no se usa IP por DHCP?

Entonces, ¿por qué la guía te dice "por DHCP, o 192.168.50.20 fija" para la máquina Cliente?
Porque en el mundo real, los clientes (nuestros celulares, laptops) casi siempre usan DHCP; es lo más cómodo. Sin embargo, en este laboratorio estás creando una red interna completamente aislada. ***A veces, en esas redes cerradas de VirtualBox, no hay ningún equipo configurado para actuar como "repartidor automático de direcciones"*** (Servidor DHCP).
Si configuras a tu cliente por DHCP y no hay nadie en esa red interna que le entregue una dirección, tu cliente se quedará sin IP, sin remitente, y el ping fallará. Por eso, la guía te da la `.20` fija como una opción infalible: es una forma de garantizar a mano que tu máquina cliente tenga su dirección lista para empezar a enviar solicitudes al servidor web.



ETAPA 1 — La segunda máquina

Objetivo: tener una segunda máquina virtual que hará de cliente. Se instala desde cero, como la primera. Este es el trabajo del primer bloque de hoy.

Su primera máquina: es la que instalaron en la sesión 5 y ya tiene Ubuntu Server. Esa será el SERVIDOR. Hoy instalan una segunda que será el CLIENTE.

1. En VirtualBox, cree una máquina nueva con los mismos valores de la sesión 5: nombre tel223-cliente, tipo Linux, Ubuntu 64-bit, 2048 MB de memoria, 2 procesadores, disco de 20 GB dinámico. Seleccione la misma imagen de Ubuntu Server.
2. Instale Ubuntu Server igual que la vez pasada. Recuerde las dos casillas que importan: teclado Spanish Latin American, y marcar Install OpenSSH server.

   Mientras instala: no espere mirando la barra. El docente va a explicar la parte de red. La instalación trabaja sola.
3. Cuando termine e ingrese, apague la máquina para configurar la red:

   ```
   sudo poweroff
   ```

Fin de la etapa 1 — su segunda máquina, tel223-cliente, está instalada. Ahora vienen los dos adaptadores.

- [x] ~~Etapa 1 - Laboratorio 1~~

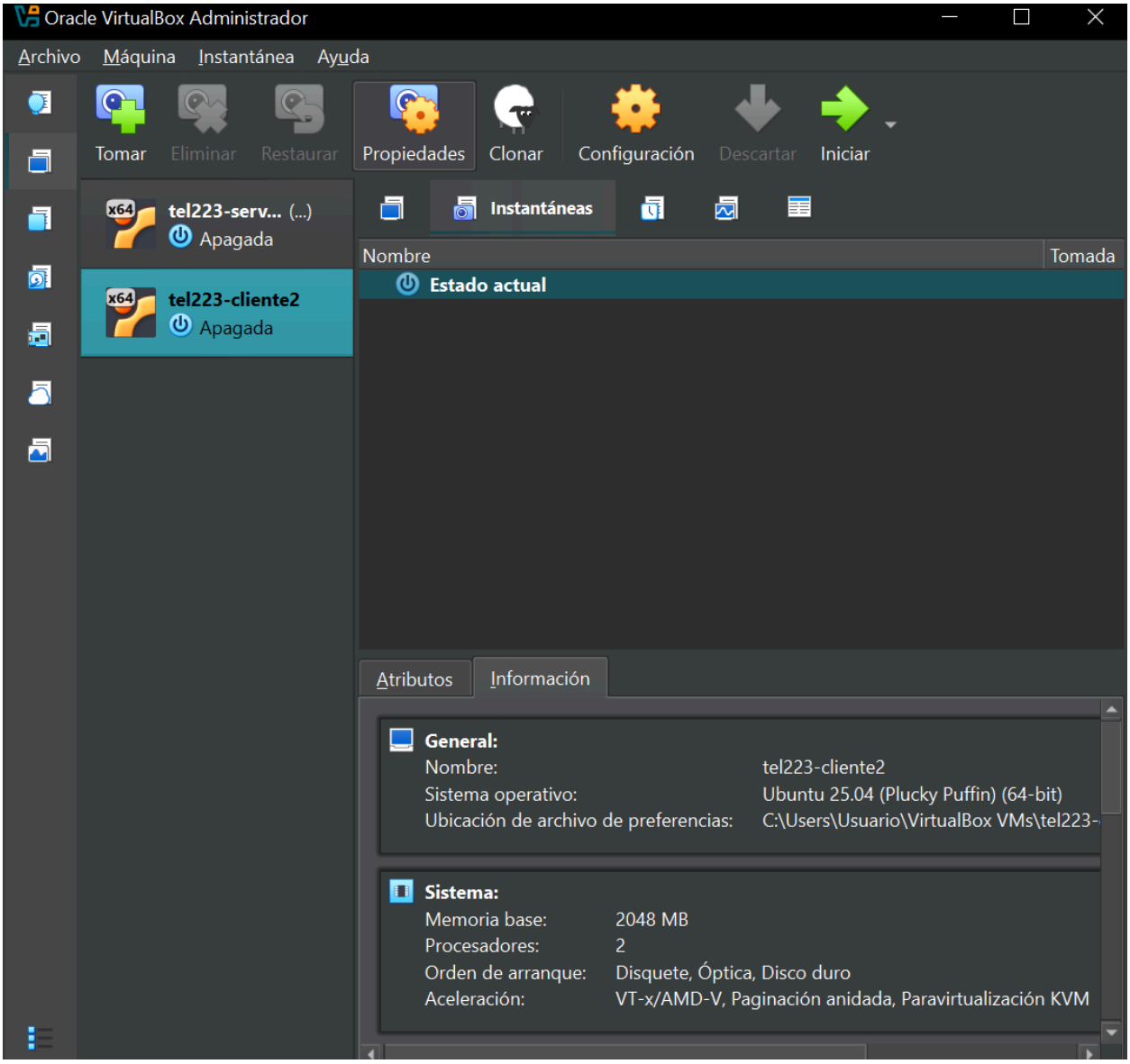
ETAPA 2

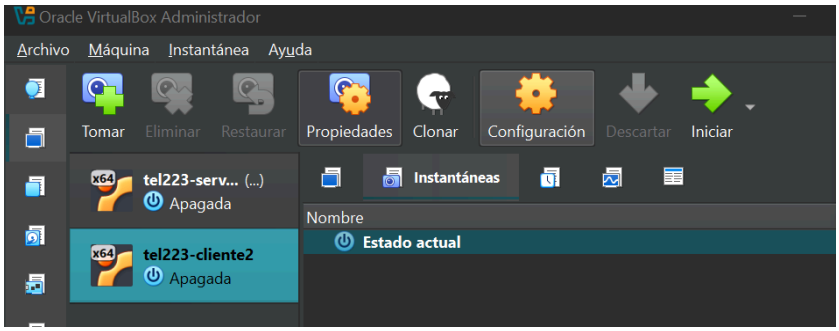
ETAPA 2 — Los dos adaptadores en las dos máquinas

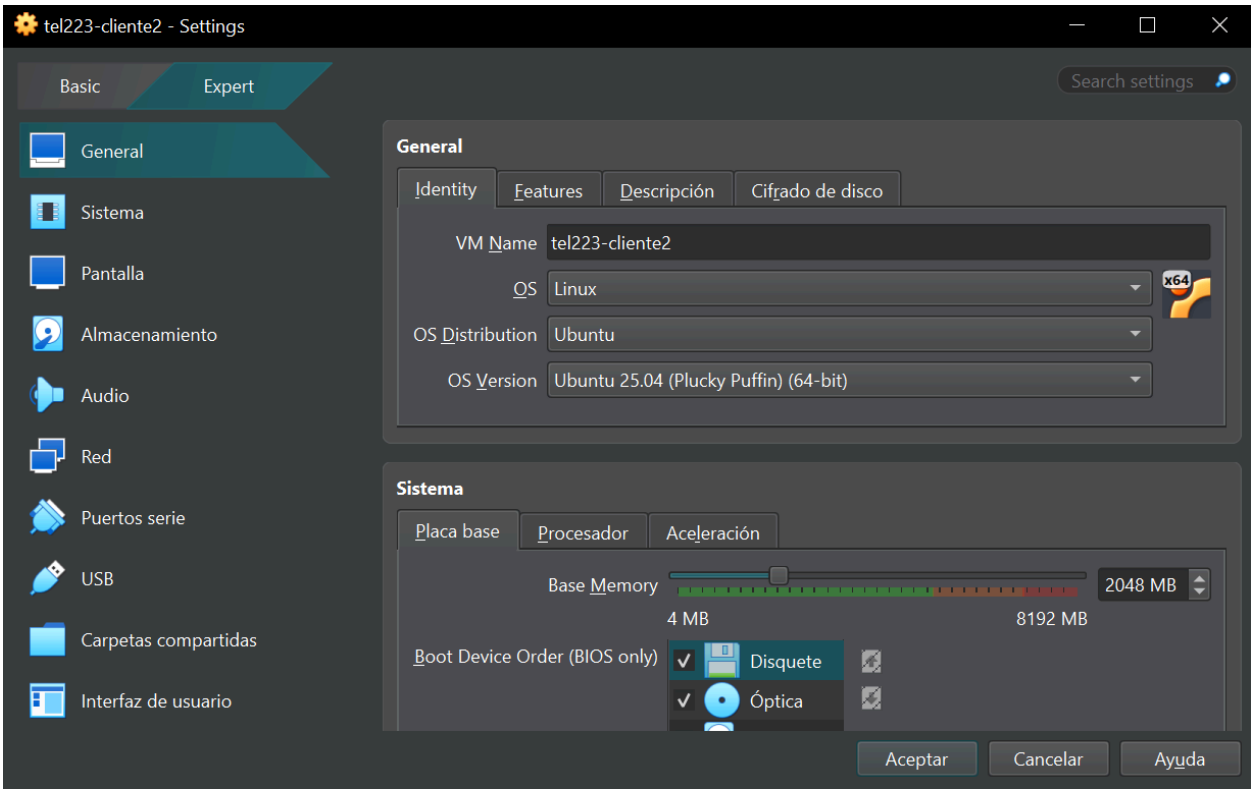
Objetivo: dejar las dos máquinas con Adaptador 1 en NAT y Adaptador 2 en la misma red interna. Con las dos máquinas APAGADAS.

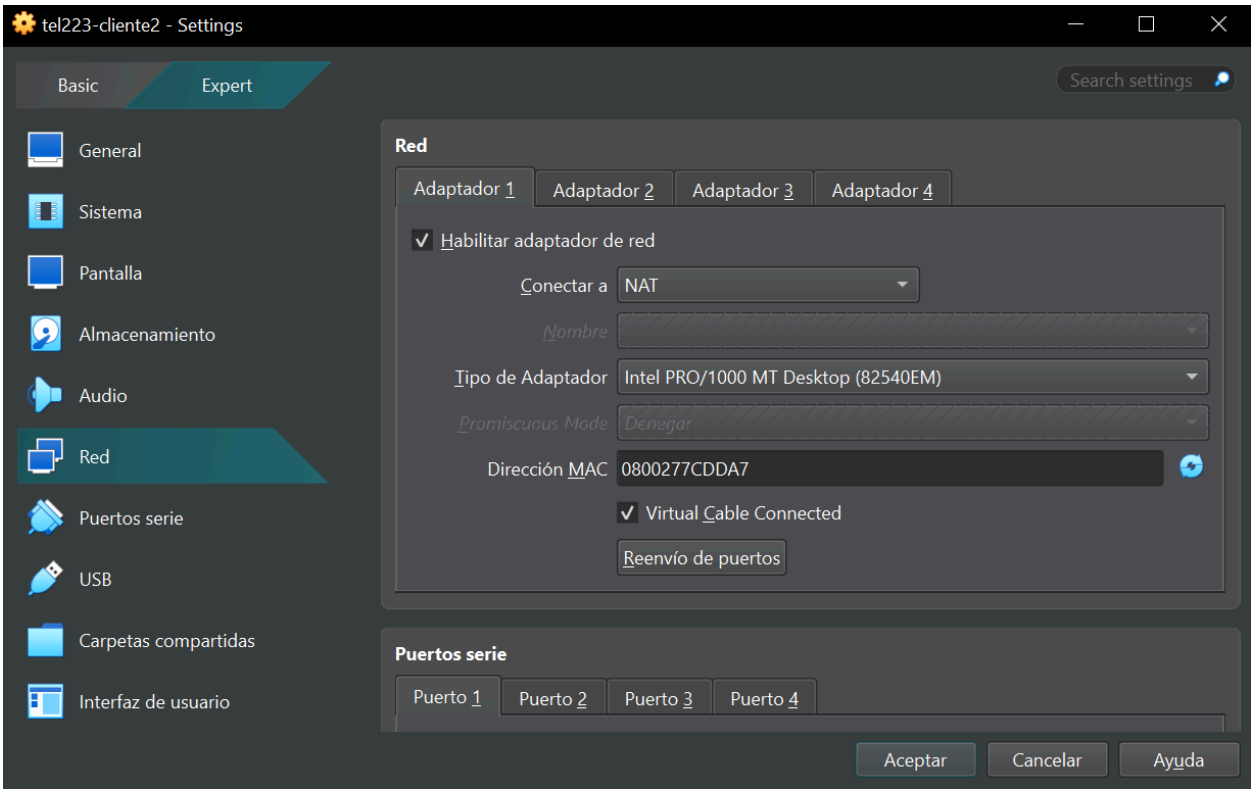
Haga esto en las DOS máquinas, la servidor y la cliente:

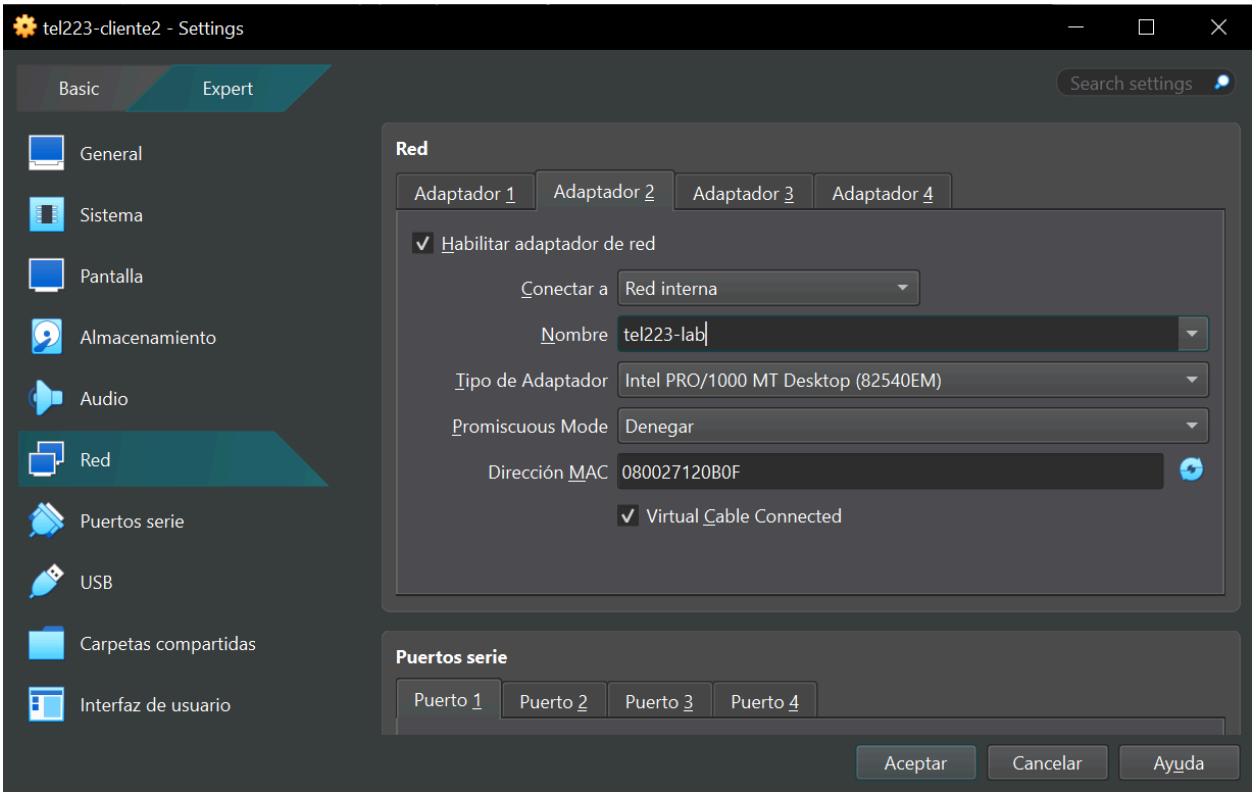
4. Seleccione la máquina, Configuración, Red. En el Adaptador 1, confirme que está habilitado y en modo NAT. No lo toque más.
5. Vaya a la pestaña Adaptador 2. Marque Habilitar adaptador de red. En Conectado a, elija Red interna. En Nombre, escriba exactamente tel223-lab. Acepte.



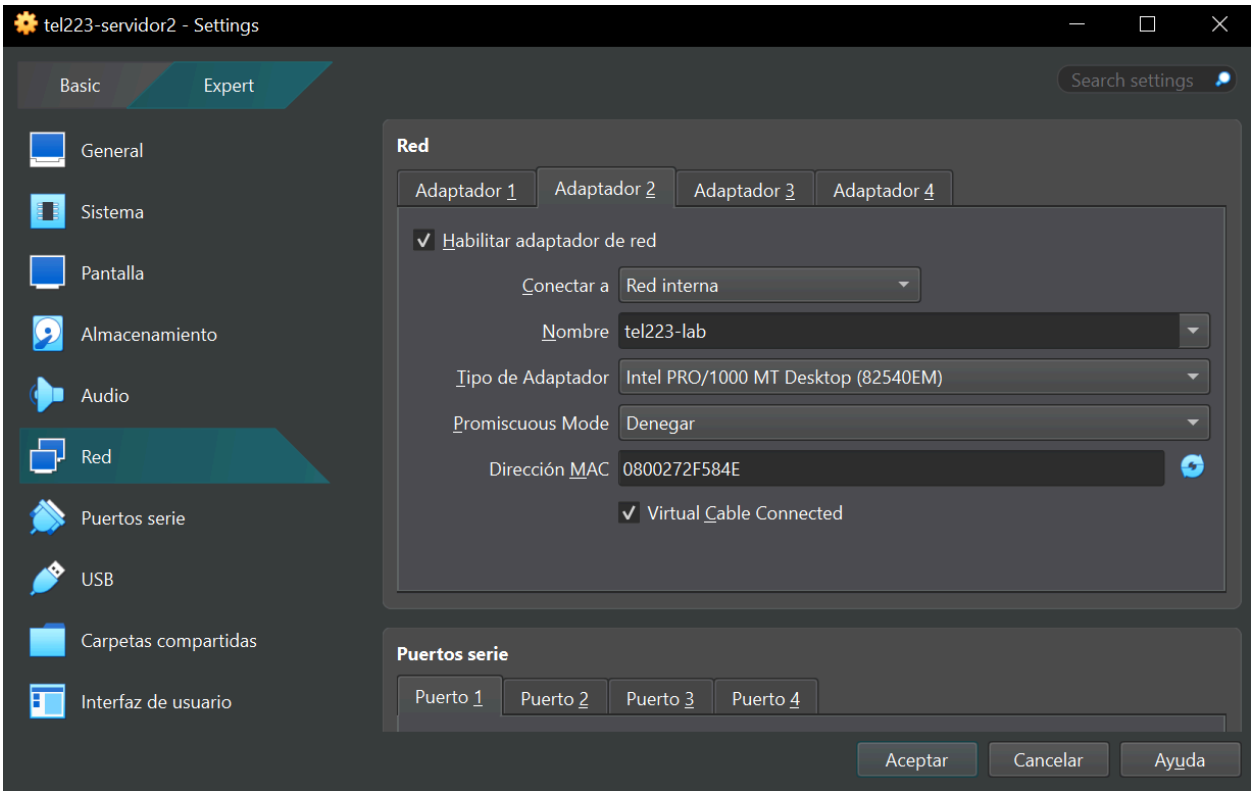

Oracle VirtualBox Administrador
Archivo
Máquina
Instantánea
Ayuda
Tomar
Eliminar
Restaurar
Propiedades
Clonar
Configuración
Descartar
Iniciar
tel223-serv... (...)
Apagada
tel223-cliente2
Apagada
Instantáneas
Nombre
Estado actual


tel223-cliente2 - Settings
Basic
Expert
Search settings
General
Sistema
Pantalla
Almacenamiento
Audio
Red
Puertos serie
USB
Carpetas compartidas
Interfaz de usuario
General
Identity
Features
Descripción
Cifrado de disco
VM Name
tel223-cliente2
OS
Linux
OS Distribution
Ubuntu
OS Version
Ubuntu 25.04 (Plucky Puffin) (64-bit)
Sistema
Placa base
Procesador
Aceleración
Base Memory
2048 MB
4 MB
8192 MB
Boot Device Order (BIOS only)
Disquete
Óptica
Aceptar
Cancelar
Ayuda





Servidor2 - Settings



Verifique esto dos veces: las dos máquinas deben tener el Adaptador 2 con el MISMO nombre de red: tel223-lab. Si una dice tel223-lab y la otra dice otra cosa, no se van a ver. Es el error más común.			
Servidor — Adaptador 2 nombre de red	tel223-lab	Cliente — Adaptador 2 nombre de red	tel223-lab

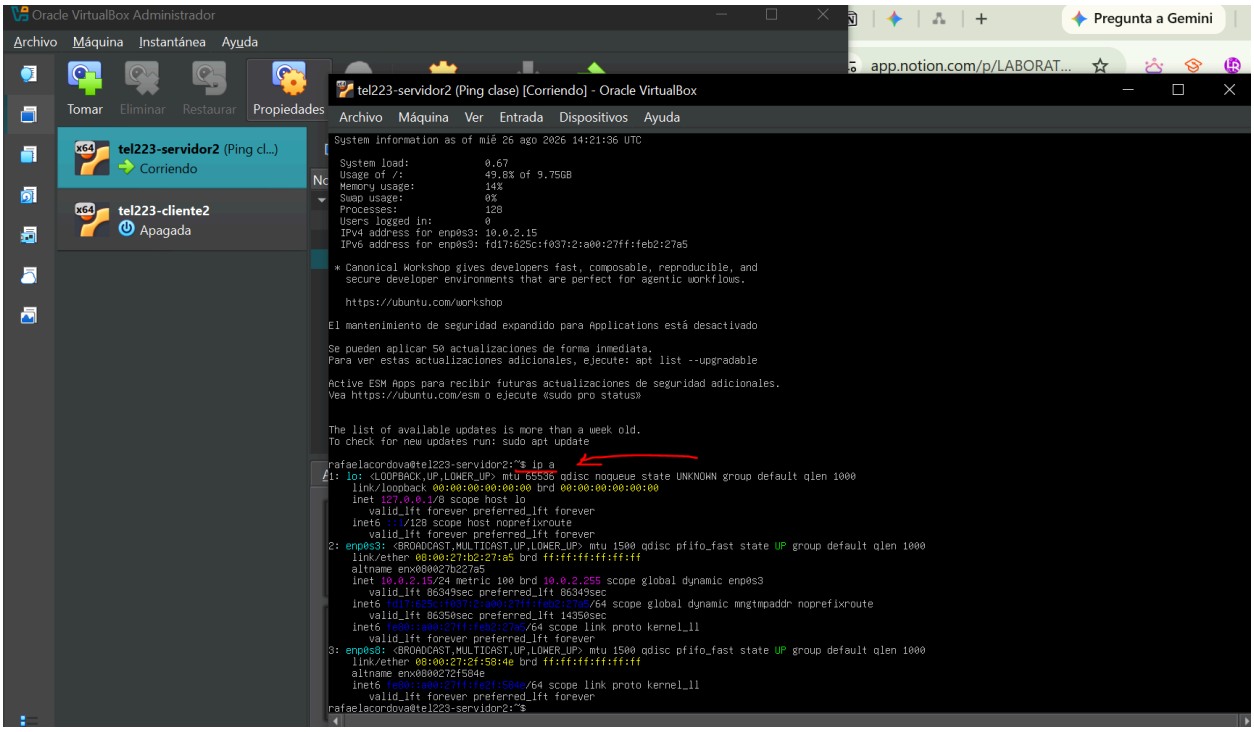
ETAPA 3

ETAPA 3 — La dirección del servidor

Objetivo: darle al servidor su dirección fija en la red interna. Encienda solo la máquina SERVIDOR.

6. Encienda la máquina servidor e ingrese. Averigüe cómo se llaman sus dos tarjetas:

```
ip a
```



NAT → ENP0S3

Red Interna → ENP0S8

▼ ¿POR QUÉ LAS IPs TIENEN ESOS NOMBRES?



En Linux moderno, estos nombres no son letras puestas al azar, sino un **mapa exacto de dónde está enchufada la tarjeta** en la placa madre virtual.
Para no memorizarlo sin sentido, solo hay que leer el nombre como si fueran unas coordenadas físicas divididas en tres partes:

- `en` **(Ethernet):** Indica el tipo de tecnología. Le dice al sistema operativo "esta es una conexión de red por cable tradicional". Si estuvieras usando una tarjeta Wi-Fi, el nombre empezaría con `wl` (Wireless).
- `p0` **(PCI bus 0):** En arquitectura de computadoras, el bus PCI es la "avenida principal" dentro de la placa madre por donde viajan

los datos. El `0` significa que la tarjeta está conectada en la avenida número cero (la principal).

- `s3` y `s8` **(Slot 3 / Slot 8):** Significa "ranura" (slot). Imagina una regleta de enchufes. Cuando el programa VirtualBox creó tu máquina virtual, decidió "enchufar" tu primer adaptador (el NAT) en la ranura número 3, y el segundo adaptador (la Red Interna) en la ranura libre número 8.

En resumen, `enp0s3` se traduce lógicamente como: *"Cable de red conectado en la avenida principal, ranura número 3"*.

▼ ¿POR QUÉ SON ETHERNET SI ES UN MUNDO SIMULADO?



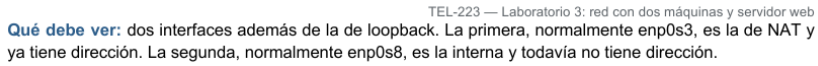
Es una duda excelente y muy lógica. Si todo es virtual y tu computadora física probablemente está conectada por Wi-Fi, parece contradictorio que la máquina virtual indique que usa una conexión Ethernet (`en`).
La lógica detrás de esto se basa en cómo funciona la **simulación de hardware** (emulación) y en mantener la **simplicidad** del sistema:

- **VirtualBox es un simulador de piezas de computadora:** Cuando creas una máquina virtual, el programa no le da a la VM acceso directo a la tarjeta Wi-Fi real de tu laptop. En su lugar, VirtualBox fabrica una placa madre "falsa" con componentes electrónicos simulados. Para la red, decide fabricar y conectar siempre una tarjeta de red alámbrica genérica.
- **Emular Wi-Fi es innecesariamente complejo:** Para simular una tarjeta inalámbrica (`wl`), VirtualBox tendría que inventar señales de radio virtuales, simular antenas, crear redes invisibles y

obligarte a configurar contraseñas de Wi-Fi adicionales dentro del servidor. Es un proceso pesado y propenso a fallas.

- **La "ilusión" del cable:** Es mucho más eficiente hacer trampa. VirtualBox toma el internet de tu computadora real (sin importar si tú estás usando el Wi-Fi de tu casa) y se lo "inyecta" a la máquina virtual simulando que están unidas por un cable de cobre de alta velocidad.
- **Lo que descubre Linux:** Cuando enciendes tu Ubuntu Server, el sistema operativo explora los componentes de su placa madre virtual. Se topa con la tarjeta que VirtualBox le creó, analiza su arquitectura y deduce: *"Esto es una tarjeta de red con puerto para cable"*. En consecuencia, la nombra `en` (Ethernet).

Para el sistema operativo de tu servidor, el mundo inalámbrico no existe. Vive en un entorno donde cree firmemente que es una gran computadora de escritorio conectada por un cable físico directo al router, lo cual garantiza que la conexión sea estable, sin cortes y sin necesidad de configurar contraseñas de Wi-Fi en cada máquina virtual que inventes.



No tiene dirección enp0s8 porque no hay inet

▼ ¿POR QUÉ ENP0S8 NO TIENE IP ASIGNADA Y NAT SI VIENE CON UNA DE FORMA AUTOMÁTICA?

💡 La diferencia radica en cómo VirtualBox administra los distintos tipos de red. Habilitar el adaptador en la Etapa 2 equivale únicamente a conectar un cable de red a la placa madre virtual; sin embargo, enchufar el cable no garantiza que la máquina reciba automáticamente un número lógico (dirección IP) para comunicarse.

Aquí tienes la lógica exacta de por qué se comportan de manera distinta:

- **Adaptador 1 (NAT):** Viene con una dirección IP automática (como la `10.0.2.15` que ves en tu consola) porque en este modo, VirtualBox actúa como el enrutador de tu casa. VirtualBox tiene un servicio interno (DHCP) encendido por defecto para el modo NAT que reparte direcciones inmediatamente para garantizar que la máquina tenga salida a Internet para instalar paquetes.
- **Adaptador 2 (Red Interna):** No tiene IP (no ves la línea `inet` bajo `enp0s8`) porque una "Red interna" es un entorno virtual completamente vacío y aislado del exterior. En este enlace privado llamado `tel223-lab` no existe ningún servicio automático repartiendo direcciones. Al encender la máquina, el sistema operativo reconoce que el cable está conectado, pero se queda esperando a que alguien le asigne un número.

El sentido lógico de la configuración manual
Todo en la arquitectura de servidores exige predictibilidad. El propósito principal de un servidor web es ofrecer un servicio en una ubicación conocida. Si el segundo adaptador recibiera su IP automáticamente de VirtualBox, esa dirección podría cambiar cada vez que apagas o reinicias la máquina, lo que impediría que el Cliente sepa a qué puerta hacerle el *ping* o pedirle la página web.
Al obligarte a configurar la red interna manualmente en la Etapa 3, el laboratorio asegura que le asignes la dirección estática `192.168.50.11` de forma permanente. Esto garantiza que el servidor siempre responderá en el mismo lugar exacto, permitiendo que la red local funcione de manera estable.

7. Edite el archivo de red:

```
sudo nano /etc/netplan/99-tel223.yaml
```

Escriba esto, usando los nombres de interfaz que acaba de ver:

```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses: [192.168.50.11/24]
```

Cuidado con dos cosas: la dirección fija va en el adaptador INTERNO, el segundo. Y la sangría se hace con espacios, nunca con tabulaciones.

▼ ¿Por qué el comando sudo nano /etc/netplan/99-tel223.yaml?



El comando `sudo nano /etc/netplan/99-tel223.yaml` se divide en tres partes lógicas que le dan una instrucción muy precisa al sistema operativo.

- `sudo` **(La llave maestra):** Tu recuerdo es perfectamente correcto. Los archivos que controlan el internet y las redes son críticos para el funcionamiento del servidor. Linux bloquea estos archivos para que los usuarios comunes o posibles atacantes no puedan desconectar el equipo por error. Al poner `sudo`, le estás diciendo al sistema: "Tengo credenciales de administrador, permíteme modificar la configuración central de la máquina".
- `nano` **(La herramienta de trabajo):** Como tu máquina virtual Servidor no tiene un entorno gráfico con ratón y ventanas (como Windows o macOS), no puedes abrir un "Bloc de notas" tradicional. `nano` es el nombre de un programa editor de texto diseñado para funcionar exclusivamente dentro de la línea de comandos. Básicamente le estás pidiendo a la máquina: "Abre el bloc de notas de la terminal".
- `/etc/netplan/99-tel223.yaml` **(La ruta y el archivo):** Esta es la dirección exacta del documento en el disco duro que vas a modificar. Se lee lógicamente de izquierda a derecha como si abrieras carpetas:

- `/etc/`: En la arquitectura de Linux, la carpeta raíz llamada `etc` es el cerebro de las configuraciones; allí se guardan todos los ajustes globales del sistema operativo.
- `netplan/`: Es la subcarpeta exclusiva que usa Ubuntu moderno para leer cómo deben comportarse las tarjetas de red físicas y virtuales.
- `99-tel223.yaml`: Es el nombre del archivo específico que vas a editar para este laboratorio. La terminación `.yaml` indica el formato en el que está escrito (por esa razón estricta de formato, la guía te advierte que la sangría de este archivo se hace siempre con espacios y nunca con tabulaciones).

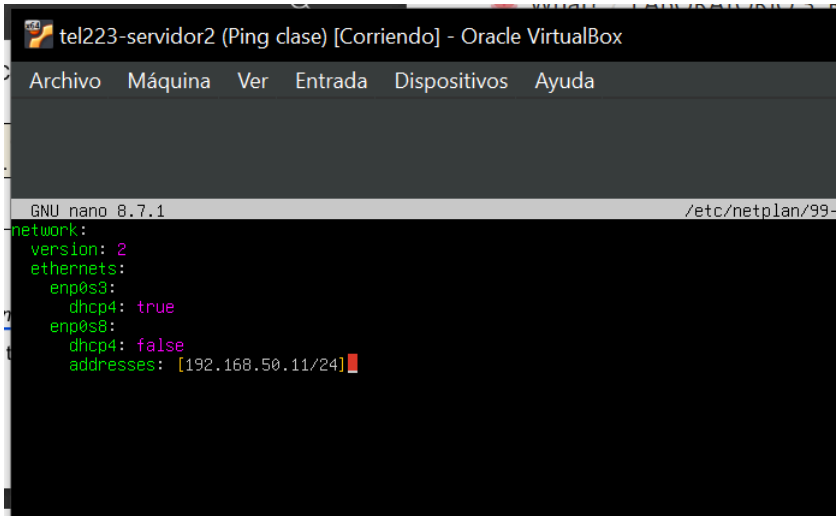
La lógica del comando completo, leída de corrido, significa: *"Como administrador (`sudo`), usa el editor de texto de la terminal (`nano`) para abrir y editar el archivo de configuración de red de la materia (`/etc/netplan/99-tel223.yaml`)"*.

▼ ¿Por qué se llama nano?



El nombre **nano** proviene de un juego de palabras científico relacionado con los prefijos del Sistema Internacional de Unidades. Este programa fue creado originalmente como una alternativa gratuita a un editor de texto más antiguo y de código cerrado llamado **Pico**. Dado que "pico" representa una unidad de medida extremadamente pequeña (la billonésima parte), el creador decidió utilizar el siguiente prefijo en la escala de magnitud para nombrar a su herramienta: "nano" (la milmillonésima parte).
Es una tradición clásica en el mundo de Linux y el software libre nombrar a los programas sucesores o clones haciendo referencias ingeniosas o bromas relacionadas con el programa original en el que se inspiraron.

En los archivos de configuración como el que muestra la imagen (que utiliza el formato YAML para Netplan), cada nivel de sangría equivale estrictamente a **2 espacios**.



▼ ¿Por qué del texto en el editor nano?

El archivo que estás editando en la imagen **"image_754a7d.png"** está escrito en un lenguaje llamado YAML. Su lógica principal es la **jerarquía**: usa las sangrías (los espacios) para crear "carpetas" o categorías de información. El sistema operativo lee este archivo de arriba hacia abajo y agrupa las instrucciones según esos espacios. Aquí tienes la explicación lógica, línea por línea, de lo que le estás ordenando a la computadora:

- `network:`
  Es el título principal del documento. Le indica al sistema operativo (específicamente a la herramienta Netplan): *"Atención, todo lo que está escrito de aquí en adelante son configuraciones de red, no de pantalla, ni de teclado"*.
- `version: 2`
  El software evoluciona y cambia su sintaxis con los años. Esta línea le dice a Netplan: *"Para entender este documento, utiliza las reglas gramaticales de la versión 2 de tu programa"*.

- `ethernets:`
  Aquí empiezas a filtrar el tipo de hardware. Como las redes pueden ser Wi-Fi (`wifis:`), puentes virtuales (`bridges:`), etc., con esta línea indicas: *"A continuación, te voy a dar las reglas exclusivas para las tarjetas de red por cable (ethernet)"*.

- `enp0s3:`
  Bajas otro nivel en la jerarquía para ser más específico. Le estás diciendo: *"De todas las tarjetas por cable que tengas, busca específicamente la que está conectada en la ranura 3 (tu Adaptador 1 / NAT). Para esta tarjeta, aplica la siguiente regla:"*.

- `dhcp4: true` `dhcp` es el servicio que reparte IPs automáticamente, y el `4` especifica que use el protocolo IPv4. Al poner `true` (verdadero), le ordenas: *"Sí, enciende el modo automático. Pídele una dirección IP al router virtual de VirtualBox para que podamos tener salida a Internet"*.

- `enp0s8:`
  Nota cómo la sangría retrocede para alinearse con `enp0s3`. Esto significa: *"Terminamos con la tarjeta anterior. Ahora busca la tarjeta conectada en la ranura 8 (tu Adaptador 2 / Red Interna) y aplícale estas reglas distintas:"*.

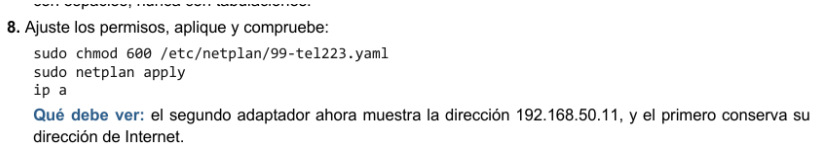
- `dhcp4: false`
  Como esta tarjeta está conectada al "pasillo privado" (`tel223-lab`) donde no existe ningún router automático, le ordenas: *"No intentes buscar una dirección IP automáticamente. Apaga esa función (`false`), porque nadie te va a responder y solo perderás el tiempo"*.

- `addresses: [192.168.50.11/24]`
  Como apagaste el modo automático en el paso anterior, el sistema te obliga a darle la dirección manualmente. Con esta línea dictas la regla final: *"Asignate a ti misma de forma permanente la dirección `192.168.50.11`"*.

- *Nota sobre el `/24`:* Ese número final es la **máscara de subred**. Le sirve a la computadora para saber el tamaño de su "vecindario". Al poner `/24`, le estás diciendo: *"Solo comunícate directamente con otras computadoras cuyas direcciones empiecen exactamente con `192.168.50.xxx`"* (lo cual incluirá a tu Cliente que tendrá la `.20`).

Para guardar los cambios en el editor `nano` y regresar a la línea de comandos, utiliza esta secuencia:

1. Presiona **Ctrl + O** para guardar los cambios.
2. Presiona **Enter** para confirmar el nombre del archivo.
3. Presiona **Ctrl + X** para salir del editor.



8. Ajuste los permisos, aplique y compruebe:

```
sudo chmod 600 /etc/netplan/99-tel223.yaml
sudo netplan apply
ip a
```

Qué debe ver: el segundo adaptador ahora muestra la dirección 192.168.50.11, y el primero conserva su dirección de Internet.

▼ ¿Cuál es el sentido del comando sudo chmod 600 /etc/netplan/99-tel223.yaml?

💡 El comando se enfoca en el concepto de la seguridad de los archivos. Como ya entiendes `sudo` y la ruta del archivo, aquí tienes la lógica exacta de las dos piezas centrales:

- `chmod` **(Change Mode / Cambiar Modo):** En Linux, cada archivo tiene un sistema de seguridad o "candado" que dicta quién puede leerlo, escribirlo o ejecutarlo. `chmod` es la instrucción que te permite cambiar los permisos de ese candado para el archivo. Al usarlo, estás cumpliendo el paso de "Ajuste los permisos".
- `600` **(La combinación del candado):** Este número no es aleatorio, está dividido en tres dígitos y cada uno representa un nivel de acceso (Dueño, Grupo, Otros):

- **El primer dígito (6)** representa los permisos del **Dueño** (en este caso, el administrador absoluto, porque usaste `sudo`). En el sistema de valores de Linux, el permiso de Lectura vale 4 y el de Escritura vale 2. Sumados dan 6. Esto significa: "El administrador puede leer y modificar este archivo".
- **El segundo dígito (0)** representa a un **Grupo** específico de usuarios. El 0 significa "absolutamente ningún permiso". No pueden ni abrir el documento.
- **El tercer dígito (0)** representa a **Otros** (cualquier otro visitante o usuario sin privilegios que entre al sistema). El 0 significa "absolutamente ningún permiso".

La lógica de por qué la guía te obliga a hacer esto:
El archivo YAML que acabas de editar tiene las configuraciones maestras de la red. Si cualquier usuario ajeno pudiera leer o modificar ese archivo, podría alterar las direcciones y desconectar el servidor, lo cual es una vulnerabilidad grave.
El programa que controla la red en Ubuntu (Netplan) está diseñado para ser muy estricto con la seguridad: si detecta que tu archivo de configuración tiene permisos demasiado abiertos o descuidados, arrojará una advertencia de seguridad y podría negarse a conectar la red. Al ejecutar este comando, estás blindando y aislando el archivo para que nadie más que el administrador pueda siquiera asomarse a ver qué direcciones IP tiene el servidor.

▼ ¿Por qué el permiso 600?

💡 Los permisos estándar en Linux se representan con tres letras individuales: **r**, **w** y **x**.

- **r (Read / Lectura):** Permite abrir y ver el contenido del archivo. Su valor numérico en el sistema es **4**.

- **w (Write / Escritura):** Permite modificar, guardar cambios o borrar el archivo. Se utiliza únicamente la letra **w** (no "wr"). Su valor numérico en el sistema es **2**.
- **x (Execute / Ejecución):** Permite correr el archivo como si fuera un programa o un script. Su valor numérico en el sistema es **1**.

La combinación de estos valores matemáticos es exactamente de donde sale el número `6` que usaste en tu comando anterior (`chmod 600`). Al sumar el permiso de lectura (**4**) con el permiso de escritura (**2**), el resultado es **6**, lo que le otorga la capacidad de leer y modificar el documento al administrador.

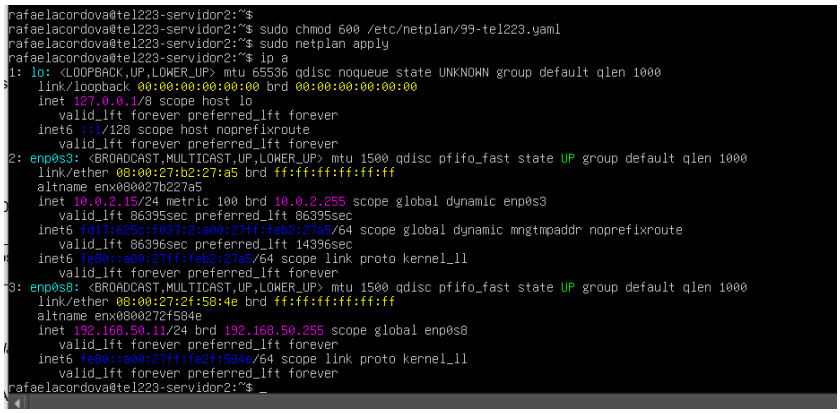
▼ Confirmación del comando sudo netplan apply



Tu deducción sobre el resultado es correcta: el comando sirve exactamente para asignarle la IP fija a esa segunda tarjeta de red para la red interna. Sin embargo, hay un pequeño matiz conceptual sobre cómo funciona.

- **El rol de nano:** `nano` es únicamente el editor de texto (como la pluma con la que escribes). Una vez que guardas el archivo y sales de `nano`, su trabajo termina. El comando no "ejecuta nano".
- **El rol de netplan apply:** `netplan` es el programa administrador de redes de Ubuntu. Al darle la orden `apply` (aplicar), le estás diciendo al sistema: *"Lee el archivo YAML que acabo de modificar, entiende las nuevas reglas y haz los cambios en las tarjetas de red de inmediato"*.

Gracias a este comando, el sistema operativo configura la nueva dirección IP en la tarjeta `enp0s8` "en caliente", sin que tengas que reiniciar toda la máquina virtual para que los cambios surtan efecto.



ETAPA 4

ETAPA 4 — El ping de prueba (el que antes fallaba)

Objetivo: confirmar que las dos máquinas se ven. Este es el punto exacto donde la vez pasada todo se detenía. Hoy va a funcionar, porque las dos están en la misma computadora.

9. Sin apagar el servidor, encienda TAMBIÉN la máquina cliente e ingrese. Ahora tiene las dos máquinas corriendo a la vez, en su propia computadora.

10. En el CLIENTE, déle una dirección a su adaptador interno. Edite su archivo de red:

```
sudo nano /etc/netplan/99-tel223.yaml
```

▼ ¿Por qué ambas máquinas deben de estar corriendo a la vez?

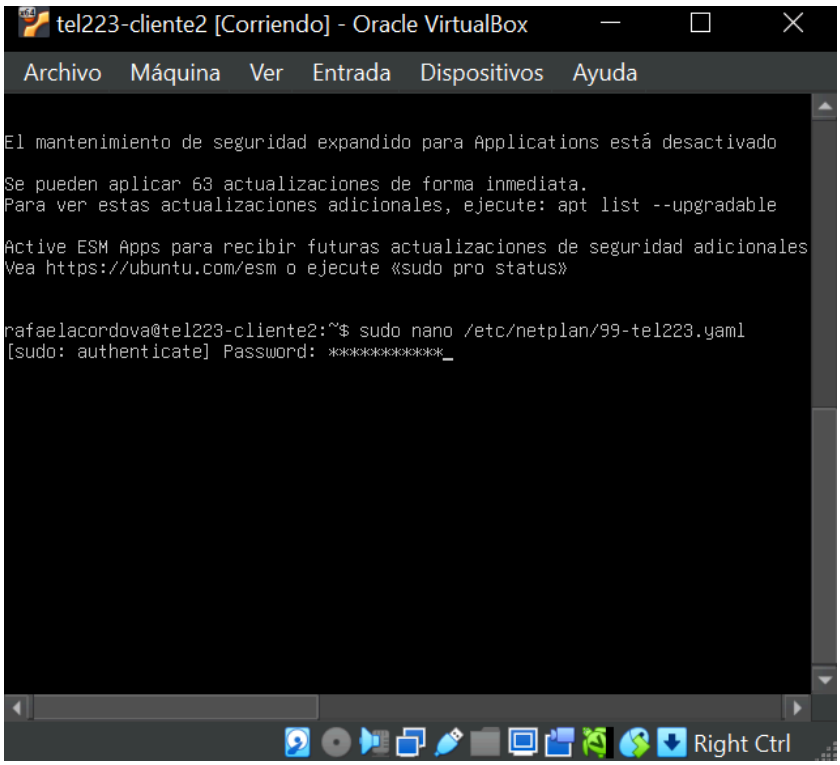


Un comando `ping` no verifica simplemente si una dirección IP está configurada o si "existe" en papel; es una solicitud de comunicación activa que exige que el equipo de destino reciba el mensaje, lo procese y construya una respuesta de vuelta.
La lógica técnica de por qué ambas máquinas virtuales deben estar encendidas simultáneamente se basa en cómo interactúan el hardware y el software:

- **Es el equivalente a una llamada telefónica:** Si marcas el número correcto de un amigo pero su celular está apagado sin batería, la llamada nunca se completará. El número es válido, pero el aparato no tiene energía para procesar la señal de radio, hacer sonar la bocina ni permitir que tu amigo conteste.

- **El ping es un "paquete de eco" (ICMP):** Cuando tu máquina Cliente hace un `ping`, empaca un mensaje de prueba y lo lanza por el cable de red virtual hacia la dirección del Servidor.
- **Ausencia de un "cerebro" escuchando:** Si la máquina Servidor está apagada, su sistema operativo Linux no está funcionando en la memoria RAM y su tarjeta de red virtual (`enp0s8`) está literalmente "desenergizada". No existe ningún programa ejecutándose que pueda escuchar la llegada de tu paquete.
- **El fracaso por espera (Timeout):** Como el Servidor está apagado y no puede construir el mensaje de respuesta ("Sí, recibí tu paquete, aquí estoy"), tu Cliente se queda esperando en el vacío hasta que se agota el tiempo límite y declara que el paquete se perdió.

Por esta razón, la red requiere que ambos extremos del canal estén alerta. Esto explica por qué el paso 9 de tu guía hace hincapié en la instrucción: "Sin apagar el servidor, encienda TAMBIÉN la máquina cliente", para garantizar que "tiene las dos máquinas corriendo a la vez". Sin dos sistemas operativos activos escuchando la red al mismo tiempo, el diálogo es físicamente imposible.



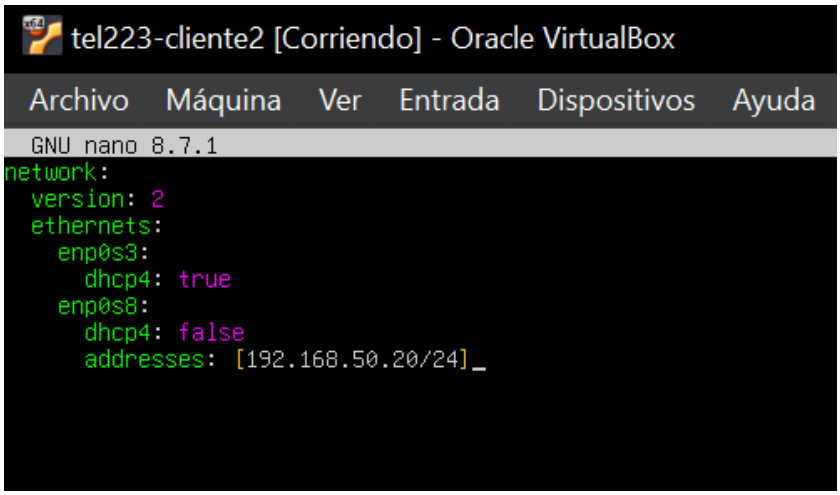
sudo nano /etc/netplan/99-tel223.yaml

Para el cliente, use la .20:

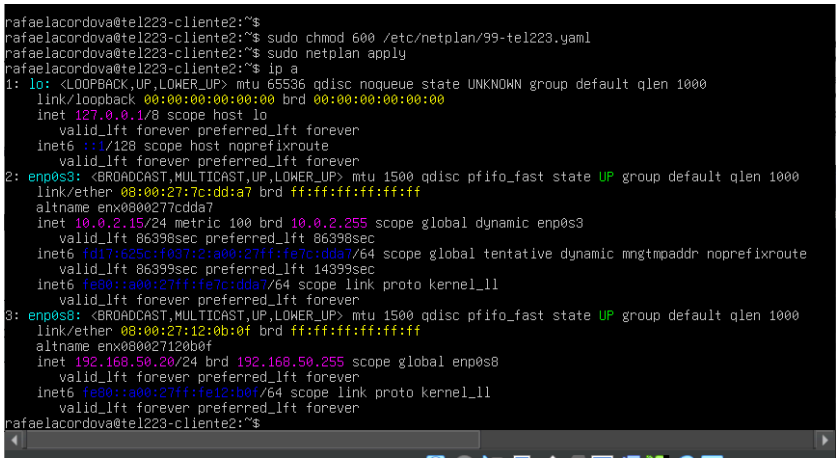
```
network:
  version: 2
  ethernets:
    enp0s3:
      dhcp4: true
    enp0s8:
      dhcp4: false
      addresses: [192.168.50.20/24]
```

11. Aplique en el cliente y verifique su dirección:

```
sudo chmod 600 /etc/netplan/99-tel223.yaml
sudo netplan apply
ip a
```



1. Presiona **Ctrl + O** para guardar los cambios.
2. Presiona **Enter** para confirmar el nombre del archivo.
3. Presiona **Ctrl + X** para salir del editor.



12. Ahora el momento de la verdad. Desde el CLIENTE, haga ping al servidor:

```
ping -c 4 192.168.50.11
```

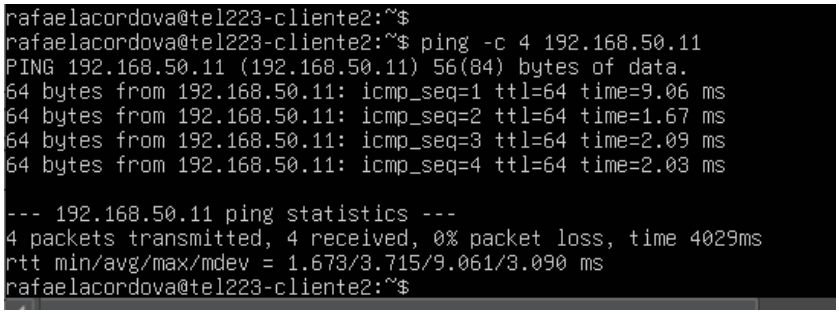
▼ ¿Por qué el comando posee -c 4?



El fragmento que falta por comprender en el comando de la imagen es `-c 4`.
La lógica de esta instrucción se basa en una diferencia clave de comportamiento entre los sistemas operativos:

- `c` **(Count / Cuenta):** Es un parámetro (o "bandera") que le indica al comando que debe llevar una cuenta de cuántos mensajes o paquetes de prueba va a enviar.
- `4`**:** Es la cantidad exacta de paquetes.

¿Por qué es necesario colocarlo?
Si haces un simple `ping` a una dirección en Windows, el sistema está programado para enviar 4 paquetes por defecto y luego detenerse solo para mostrarte el resumen.
Sin embargo, en Linux (el sistema de tu Servidor y Cliente), el comando `ping` está diseñado para ejecutarse de forma **infinita**. Si solo escribieras `ping 192.168.50.11`, la pantalla se llenaría de líneas enviando mensajes sin parar para siempre, y tendrías que interrumpirlo forzosamente presionando `Ctrl + C`.
Al añadir `-c 4`, le estás dando a Linux la instrucción lógica de comportarse de manera controlada: *"Haz un ping a esta dirección, pero limítate a enviar exactamente 4 paquetes y detente automáticamente"*. De esta forma obtienes tu resultado de conexión de forma limpia, rápida y sin bloquear tu línea de comandos.



PUNTO DE CONTROL — si ve cuatro respuestas con 0% de pérdida de paquetes, la red funciona. Esto es lo que la vez pasada no se podía lograr, y ahora sí, porque las dos máquinas están en el mismo equipo.

Si el ping falla: revise tres cosas, en este orden. Uno: que las dos máquinas tengan el Adaptador 2 con el nombre tel223-lab. Dos: que las direcciones sean .11 y .20 con la misma máscara /24. Tres: que aplicó netplan en ambas. Llame al docente si persiste.

El ping funcionó	☑ Sí ☐ No	Pérdida de paquetes	0 %

ETAPA 5

ETAPA 5 — El servidor web

Objetivo: publicar una página en el servidor. Con la red funcionando, esto es directo. Trabaje en la máquina SERVIDOR.

13. Instale el servidor web:

```
sudo apt update
sudo apt install -y nginx
```

Qué acaba de instalar: nginx, un servidor web. Desde ya está funcionando: entrega una página de bienvenida por omisión.

▼ ¿Por qué del comando sudo apt update?



`apt update` **no** descarga ningún paquete de programas ni instala software en tu máquina. Lo único que descarga de los servidores físicos de Ubuntu es texto puro: un inmenso catálogo o directorio actualizado.
Para que la lógica tenga sentido, imagina que tu servidor Ubuntu es un cliente que necesita hacer un pedido por catálogo. El proceso funciona así:

- **Los repositorios (El servidor real):** Tu máquina usa su Adaptador NAT con salida a internet para conectarse a unas supercomputadoras físicas llamadas "repositorios". Estos son servidores gigantes mantenidos por la empresa creadora de Ubuntu (Canonical) que almacenan todo el software seguro, oficial y verificado del mundo Linux.
- **El índice (El catálogo):** Lo que el servidor real te devuelve cuando haces `apt update` es literalmente una base de datos en formato de texto. Este índice contiene millones de líneas con los detalles clave de cada programa: el nombre exacto, qué versión

es la más reciente, qué otros programas necesita para funcionar (dependencias) y la dirección web (URL) exacta de dónde descargarlo. No pesa casi nada porque son solo letras, números y enlaces.

- **La interpretación de la VM:** Tu máquina virtual recibe este archivo de texto inmenso y lo guarda en silencio en una de sus carpetas internas (`/var/lib/apt/lists/`). En este punto, tu computadora todavía no tiene los programas instalados, pero ahora **sabe exactamente qué software existe en el mundo y dónde está escondido**.
- **La acción real (`apt install`):** Cuando en tu siguiente comando escribes `sudo apt install nginx`, tu máquina no sale a internet a buscar a ciegas en Google. Primero abre ese catálogo de texto local que acaba de actualizar, busca la palabra *nginx*, lee su enlace de descarga, y **solo entonces** se conecta a esa dirección de internet específica para traer el programa físico hacia tu disco duro e instalarlo.

La lógica fundamental de separar la actualización del catálogo (`update`) de la instalación real (`install`) es la eficiencia y la prevención de errores. Si el comando `update` no existiera, tu máquina intentaría leer el catálogo viejo que venía pregrabado desde el día en que se creó la imagen de instalación de VirtualBox. Si *nginx* cambió de servidor o actualizó su versión hace dos meses, tu máquina intentaría descargar el programa desde un enlace web muerto y arrojaría un error 404 Not Found.

▼ ¿Por qué el -y del comando sudo apt install -y nginx?

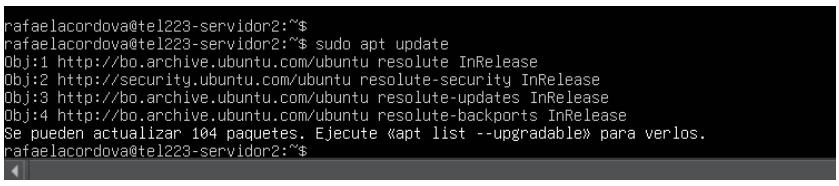


El parámetro `-y` significa simplemente **"Yes"** (Sí). Su lógica se basa en la automatización y la eficiencia al momento de instalar programas en un servidor.

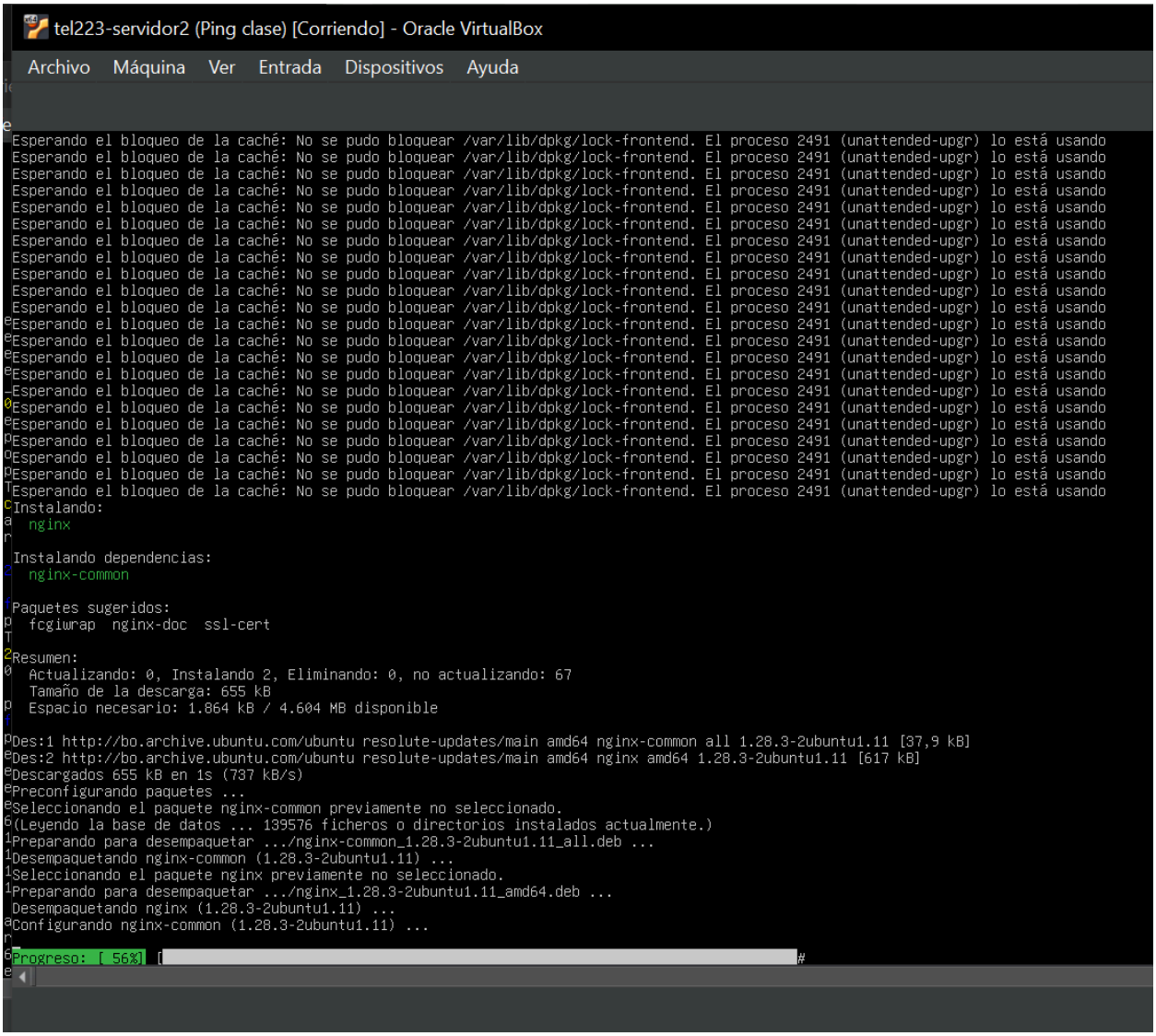
Para entender el porqué de su existencia, hay que observar cómo se comporta el gestor de paquetes normalmente:

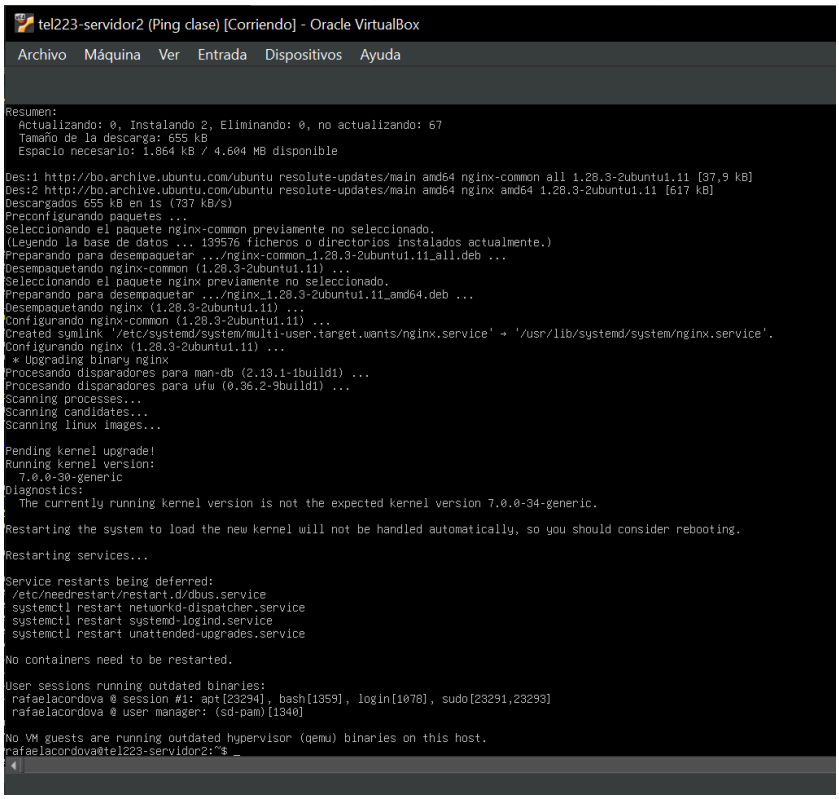
- **Sin el `y`:** Cuando le ordenas a Linux que instale un programa (como `sudo apt install nginx`), el sistema operativo está diseñado para ser muy precavido. Primero calcula cuánto va a pesar el programa, revisa si necesita descargar herramientas secundarias (dependencias) y luego **pausa todo el proceso**. Te muestra un resumen en la pantalla y te hace una pregunta obligatoria: *"Se utilizarán 5 MB de espacio adicional en el disco. ¿Desea continuar? [Y/n]"* (Yes o No). La terminal se queda totalmente congelada esperando a que un humano presione la tecla "Y" y luego Enter para confirmar.
- **Con el `y`:** Al agregar esta letra al comando, le estás dando tu consentimiento por adelantado. Básicamente le ordenas a la máquina: *"Ya sé lo que estoy instalando y autorizo el espacio que vas a usar. No te detengas a preguntarme nada a mitad de camino, responde 'Sí' automáticamente a cualquier confirmación y simplemente instala el programa"*.

En el mundo real de la ingeniería y los servidores, los administradores suelen instalar docenas de programas de golpe utilizando archivos de texto automatizados (scripts). Si no existiera el parámetro `-y`, las instalaciones automáticas fracasarían porque la máquina se quedaría bloqueada en el primer programa, esperando eternamente a que un humano confirme la descarga. Con el `-y`, el proceso se vuelve directo e ininterrumpido.



```
rafaelacordova@tel223-servidor2:~$ sudo apt install -y nginx
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando
Esperando el bloqueo de la caché: No se pudo bloquear /var/lib/dpkg/lock-frontend. El proceso 2491 (unattended-upgr) lo está usando... 15s
```





sudo loadkeys us

14. Publique su propia página. Reemplace la de bienvenida:

```
echo "<h1>Servidor TEL-223 - Puesto __</h1><p>Hola desde el servidor.</p>" | sudo tee /var/www/html/index.html
```

▼ Lógica del comando echo""...



Este comando es una técnica muy elegante en Linux llamada "pipeline" (tubería). Se utiliza para escribir texto directamente dentro de un archivo sin necesidad de abrir un editor como `nano`.
Aquí tienes la lógica de cada pieza y cómo fluye la información de izquierda a derecha:

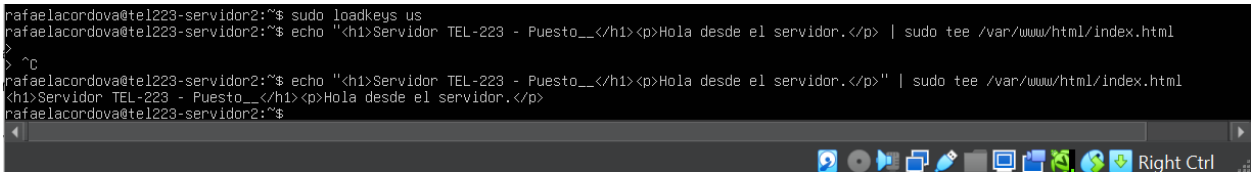
- `echo`: Funciona exactamente como su nombre indica (un eco). Su única tarea lógica es tomar el texto o código HTML que pusiste entre comillas y "gritarlo" o imprimirlo como salida.

- `|` **(Tubería o Pipe):** Este símbolo es literalmente un tubo de plomería virtual. En lugar de que el "grito" del `echo` salga hacia tu pantalla, el tubo atrapa ese texto HTML y lo envía directamente como material de entrada para el siguiente comando.
- `tee`**:** Su nombre proviene de las tuberías de fontanería en forma de "T" (T-junction). Lo que hace es tomar el texto HTML que viene viajando por el tubo y dividirlo en dos caminos: muestra una copia en tu pantalla (para que veas que funcionó) y guarda otra copia directamente dentro de un archivo de texto. Al combinarse con `sudo`, le das el poder de administrador para inyectar ese texto en carpetas protegidas del sistema.

¿Por qué cambió la dirección y ya no usamos `/etc/`?
La lógica detrás de esto se basa en cómo Linux organiza su disco duro, separando estrictamente la administración del contenido público:

- **La carpeta `/etc/` (Etcétera/Configuraciones):** Es la sala de máquinas o el cuarto de control de Linux. Aquí solo se guardan archivos de **configuración del sistema** (como tu archivo de Netplan). Nunca debes poner páginas web aquí porque los visitantes no tienen (ni deben tener) acceso a la sala de máquinas.
- **La carpeta `/var/` (Archivos Variables):** Esta sección del disco duro está destinada a datos que crecen o cambian constantemente y que son operados por los programas, no por el núcleo del sistema.
    - `/var/www/html/`: Es la convención universal en los servidores Linux para designar la "vitrina pública". Cuando instalaste el programa `nginx`, este se configuró automáticamente para mirar **única y exclusivamente** dentro de esta carpeta.
    - `index.html`: Es el nombre estándar mundial para la página principal de cualquier sitio web.

En resumen, la lógica completa del comando dice: *"Genera este código HTML (`echo`), pásalo por un tubo (`|`), y con permisos de administrador (`sudo`), usa una intersección en T (`tee`) para sobreescribir el archivo público principal (`/var/www/html/index.html`) que nuestro servidor web nginx le muestra a los visitantes"*.

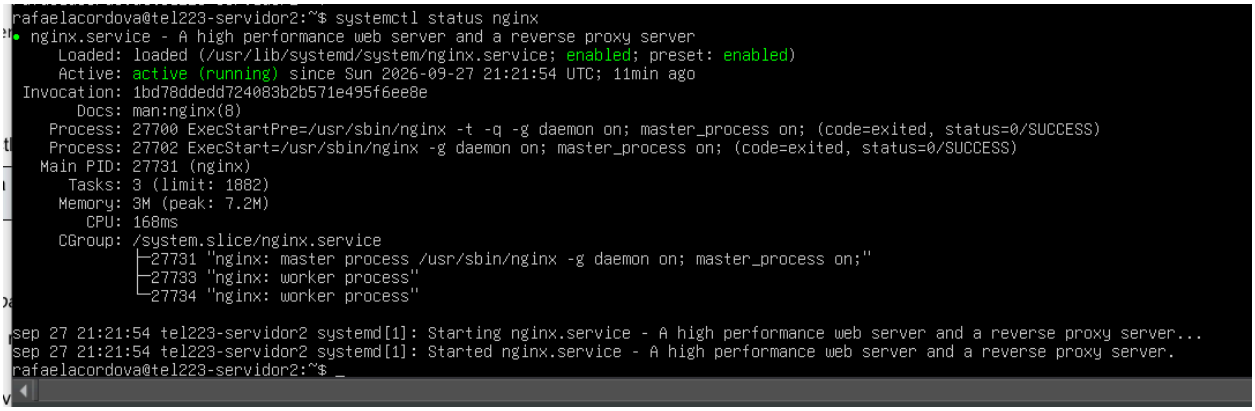


15. Compruebe que el servicio está activo:

```
systemctl status nginx
```

Qué debe ver: active (running). Si dice failed, lea el registro con journalctl -u nginx -n 20.

Fin de la etapa 5 — su servidor está publicando una página. Ahora vamos a leerla desde el cliente, y a ver la conversación HTTP por dentro.



ETAPA 6 - Leer la conversación HTTP

ETAPA 6 — Leer la conversación HTTP

Objetivo: pedir la página desde el cliente y leer la transacción completa. Trabaje en la máquina CLIENTE.

16. Desde el cliente, pida la página al servidor y lea solo las cabeceras de la respuesta:

```
curl -I http://192.168.50.11
```

Qué debe ver: la primera línea con HTTP/1.1 y el código 200 OK, y varias cabeceras: Server, Content-Type, Content-Length, Date. Esa es la respuesta del servidor.

▼ Lógica del comando curl -I http://192.168.50.11



El comando `curl -I [http://192.168.50.11](http://192.168.50.11)` funciona como un navegador web de texto que pide la página a tu servidor para leer su transacción.
La lógica de cada componente se desglosa de la siguiente manera:

- `curl` **(Client URL):** Es el programa de software. Como tu máquina virtual Cliente no tiene un entorno gráfico con navegadores como Google Chrome o Firefox, `curl` es la herramienta de la línea de comandos que se utiliza para visitar sitios web, enviar datos o descargar archivos desde internet.
- `I` **(i mayúscula):** Es la instrucción de "solo cabeceras" (Headers). Normalmente, cuando un navegador pide una página, el servidor envía un "sobre" con metadatos técnicos (las cabeceras) y dentro de ese sobre viene el cuerpo de la página (el código HTML). La letra `I` mayúscula le ordena al programa: *"Solo quiero inspeccionar los metadatos del sobre para ver si todo está bien, no descargues el cuerpo de la página web"* (por eso la guía dice "lea solo las cabeceras de la respuesta").
- `http:` **(Protocolo de Transferencia de Hipertexto):** Es el "idioma" oficial de la web. Le dice a la máquina cliente exactamente qué reglas de comunicación debe usar. Es necesario especificarlo porque `curl` es políglota; podría haber usado otros idiomas como `ftp:` (para descargar archivos pesados) o `https:` (si la página estuviera encriptada).
- `//` **(Doble barra diagonal):** Es un separador estructural inventado en los orígenes de la web. Los dos puntos (`:`) indican que terminó el nombre del protocolo, y la doble barra (`//`) le avisa al sistema operativo: *"Lo que viene a continuación no es una ruta de una carpeta local dentro de mi disco duro, sino la dirección de una computadora externa en la red"*. Marca la frontera entre el idioma que vas a usar (`http:`) y el destino físico con el que te vas a comunicar (`192.168.50.11`).

▼ Cuál es el sentido de usar -I y no -i? Cuándo uso cada uno?



La diferencia entre usar una letra mayúscula o minúscula se entiende mejor si pensamos en cómo funciona el sistema de correos al entregarte un paquete en tu casa.

1. El comando `-I` (mayúscula): Leer solo la etiqueta exterior

Al usar la mayúscula, le estás diciendo al cartero: *"Muéstrame solo la etiqueta pegada por fuera de la caja, no la abras"*.

- **La lógica de usarlo:** Se usa para ahorrar tiempo, esfuerzo y datos de internet. Imagina que encargaste una enciclopedia de 20 tomos muy pesada. No necesitas abrir la caja y sacar todos los libros a la calle para comprobar que el paquete llegó. Con solo leer la etiqueta exterior, ya confirmas quién te lo envía, que el estado es "Entregado" y que pesa los 20 kilos correctos. En la computadora, esto te permite confirmar en un segundo que una página web existe y funciona, sin tener que descargar inútilmente todas sus fotos, videos y textos.

2. El comando `-i` (minúscula): Leer la etiqueta y abrir la caja

Al usar la minúscula, le dices al cartero: *"Revisemos la etiqueta de afuera, pero también abre la caja para poder sacar y revisar todo lo que viene adentro"*.

- **La lógica de usarlo:** Lo necesitas cuando saber que el paquete "llegó" no es suficiente; necesitas comprobar el producto en sí. Si pediste un teléfono y la etiqueta dice "Electrónicos" (lo cual es correcto), no sabrás si te enviaron el modelo equivocado hasta que abras la caja y veas el aparato con tus propios ojos. En tu computadora, usas esto cuando necesitas ver exactamente qué código o texto te está entregando la página web porque sospechas que la información que contiene está mal escrita.

En resumen:

Eliges `-I` **(solo la etiqueta)** cuando tienes prisa y solo quieres confirmar si la dirección existe y si el servidor está despierto para responderte.
Eliges `-i` **(etiqueta + contenido)** cuando tienes el tiempo para descargar toda la página web porque necesitas leer el texto que te enviaron para ver si tiene algún error.

▼ Ejemplo de uso (real de respuesta HTML) del comando de -i



Imagina que ejecutas el comando `curl -i [http://192.168.50.11](http://192.168.50.11)` (con la minúscula para "abrir la caja") en una página de tu universidad que debería mostrar la lista de estudiantes inscritos.
En tu pantalla aparecería exactamente esto:
Plaintext

```
HTTP/1.1 200 OK
Server: nginx/1.28.3 (Ubuntu)
Content-Type: text/html
Content-Length: 104
Date: Sat, 03 Oct 2026 10:40:00 GMT

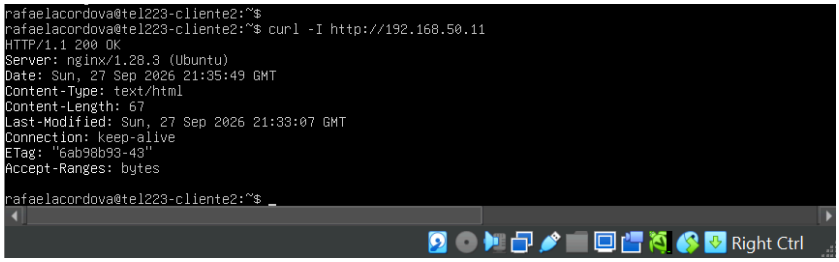
<!DOCTYPE html>
<html>
<body>
    <h1>Página de Estudiantes</h1>
    <p>Error: base de datos desconectada.</p>
</body>
</html>
```

Si divides esta respuesta en las dos partes que ya conoces, la lógica de haber usado la minúscula se vuelve evidente:

- **La etiqueta exterior (Las cabeceras de arriba):** El servidor te responde con un brillante `200 OK`. Te está diciendo: *"Todo salió perfecto, procesé tu pedido y aquí tienes la página web que me pediste"*.

- **El contenido de la caja (El código HTML de abajo):** Al leer el texto dentro de las etiquetas, descubres que la página dice `Error: base de datos desconectada`.

Si en este escenario hubieras utilizado el comando `-I` **(mayúscula)** por tener prisa, la terminal solo te habría mostrado el `200 OK`. Habrías cerrado tu computadora pensando que el sistema de la universidad funcionaba perfectamente.
Al utilizar `-i` **(minúscula)**, descubres la mentira: el servidor web de nginx hizo su trabajo y te entregó una página con éxito, pero la información *dentro* de esa página revela que hay un problema grave en otro lado del sistema. Ver el código HTML era la única forma de descubrir que el usuario final estaba viendo un mensaje de error en su pantalla.



17. Ahora pida la página completa, con cabeceras y cuerpo:

```
curl -i http://192.168.50.11
```

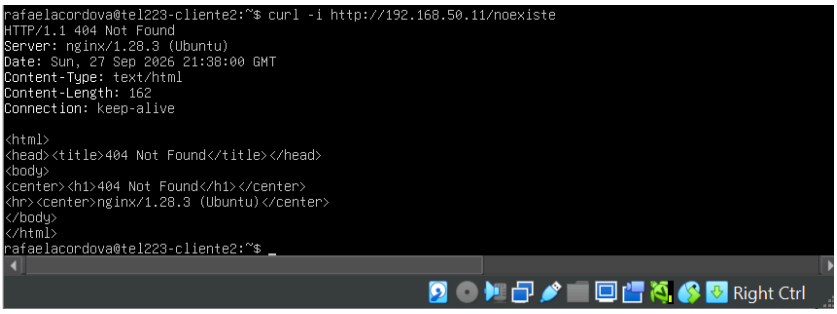
Qué debe ver: las mismas cabeceras y, debajo de una línea en blanco, el HTML de su página. Esa línea en blanco separa las cabeceras del cuerpo.



18. Provoque un error para ver otro código. Pida una página que no existe:

```
curl -i http://192.168.50.11/noexiste
```

Qué debe ver: el código 404 Not Found. El servidor respondió — no está caído — y le dice que ese recurso no existe.



▼ Lógica de las cabeceras



El sentido fundamental de esta respuesta completa es evitar el "silencio" en las comunicaciones de red. Cuando pides una página que no existe (`/noexiste`), si el servidor simplemente no respondiera, tu computadora cliente se quedaría esperando infinitamente sin saber si el cable se cortó o si el equipo se apagó. El servidor tiene la obligación técnica de enviar un paquete diciendo: *"Estoy vivo, pero lo que pides no está aquí"*.
Para lograr esto de forma ordenada, la respuesta se divide en dos partes: **Las Cabeceras** (el sobre con metadatos técnicos para la computadora) y **El HTML** (el cuerpo visual para el humano), separadas por una línea en blanco obligatoria.

Las Cabeceras (El "Sobre Técnico")

Cada línea aquí tiene un propósito técnico estricto para que la máquina cliente sepa cómo procesar la información.

- `HTTP/1.1 404 Not Found`:
  - **HTTP/1.1:** Es la versión del "idioma" que están hablando. En internet, ambas máquinas deben confirmar el dialecto exacto. La versión 1.1 inventó la capacidad de mantener un cable abierto para descargar múltiples cosas a la vez, por lo que es vital declararlo al inicio.
  - **404:** Es el código numérico del estado. Las computadoras procesan números mucho más rápido que texto, por lo que el

servidor usa el 404 para indicarle matemáticamente al navegador "el archivo no existe en mi disco duro".

    - **Not Found:** Es la traducción humana del código 404.

- `Server: nginx/1.28.3 (Ubuntu)`:
    - `nginx` es el programa informático que actúa como recepcionista. El sistema operativo Ubuntu por sí solo no sabe entregar páginas web; se instala `nginx` específicamente para que se siente en el puerto de red a escuchar y despachar archivos HTML.
    - El servidor avisa qué recepcionista te está atendiendo y su versión exacta. Para un administrador de redes, esto es vital para depurar problemas (si algo falla, sabes exactamente qué versión de software buscar en los manuales).
- `Date: Sun, 27 Sep 2026 21:38:00 GMT`: El reloj exacto en que se generó la respuesta. Si hay lentitud en la red y este paquete llega a tu computadora 5 horas tarde, tu máquina cliente usa esta fecha para saber que es información vieja y evitar errores de sincronización.
- `Content-Type: text/html`: Le dice a tu navegador qué tipo de material viene adentro. Si dijera `image/jpeg`, tu navegador intentaría dibujar una foto. Al decir `text/html`, el navegador sabe que debe prepararse para interpretar bloques de texto estructurado.
- `Content-Length: 162`: El tamaño exacto del mensaje (162 bytes). La lógica aquí es evitar atascos: la máquina cliente lee este número y sabe exactamente en qué byte terminará el mensaje, permitiéndole cortar la lectura sin quedarse esperando más datos fantasma.
- `Connection: keep-alive`: El servidor le dice al cliente: *"Dejaré el canal de comunicación abierto por unos segundos más. Si vas a pedirme otra página inmediatamente, usamos este mismo canal y nos ahorramos el tiempo de reconectarnos desde cero"*.

El Cuerpo HTML (El Mensaje Visual)

El lenguaje HTML funciona bajo una lógica de "cajas dentro de cajas". Las etiquetas (las palabras entre los signos `< >`) son instrucciones de dibujo para el navegador web.
La **barra diagonal (`/`)** es la regla más importante de la lógica HTML: significa "cierre de caja". Le indica a la computadora exactamente dónde termina un bloque. Si abres una instrucción para poner un texto en negrita y nunca pones la barra de cierre, toda la página web y el resto del internet se volvería negrita a partir de ese punto. La barra pone el límite.

- `<html>` **y** `</html>`: Son la caja contenedora principal. Le dicen al navegador: *"Todo lo que hay desde la apertura hasta el cierre con barra es la página web"*.
- `<head>` **y** `</head>`: Es la "cabeza" del documento. Aquí van las configuraciones invisibles para la pantalla.
- `<title>404 Not Found</title>`: Todo lo que va entre estas etiquetas no se dibuja en la página, sino que es el texto que aparece arriba, en la pestaña de tu navegador (como cuando dice "Facebook" o "YouTube" en la pestaña).
- `<body>` **y** `</body>`: El "cuerpo". Todo lo que está dentro de esta caja es lo que realmente se dibuja y ves en la pantalla blanca de la ventana.
- `<center>` **y** `</center>`: Una instrucción estética. Le dice al navegador que el texto de adentro no debe ir alineado a la izquierda, sino perfectamente al centro de la pantalla.
- `<h1>` **y** `</h1>`: (Heading 1). Significa Título Principal. Le ordena al navegador dibujar ese "404 Not Found" con la letra más grande y gruesa posible.
- `<hr>`: (Horizontal Rule). Es una de las pocas etiquetas que no necesita cierre. Su única función es dibujar una línea gris

horizontal de lado a lado de la pantalla para separar el título del resto.

La firma final de nginx en la pantalla
En la última línea visible de la página, nginx inyecta automáticamente `<center>nginx/1.28.3 (Ubuntu)</center>`. Esto imprime su nombre y versión en la pantalla blanca del navegador que ve el usuario.
¿Por qué es importante leer esto como ingeniero? Porque en el mundo de la infraestructura de telecomunicaciones y redes, esto se considera una fuga de información de seguridad. Nginx lo pone ahí por cortesía por defecto, pero como administrador, te interesa saber que tu servidor está "hablando de más". Si un hacker informático ve en la pantalla que estás usando exactamente la versión `1.28.3` de Ubuntu, no tiene que adivinar: irá directamente a buscar las vulnerabilidades específicas de esa versión para atacar tu servidor. Ver eso en el código fuente te confirma que debes configurar tu servidor para que censure su firma y sea más seguro.

▼ Video de una respuesta HTML/1.0

https://docs.google.com/videos/d/1XTpwCzEMhLgdUbWm840TickoLYNhEN60kvoxQG84g30/play?usp=sharing

▼ Lógica de comunicación del servidor para uso de HTML mediante otro adaptador

💡 La lógica de cómo funcionan los adaptadores en VirtualBox opera de manera muy similar a las puertas de seguridad de un banco.
La lógica del adaptador NAT (`enp0s3`)
La dirección IP que te otorga este adaptador (como la `10.0.2.15` que viste antes) es una ilusión local construida por VirtualBox. El modo NAT (Traducción de Direcciones de Red) funciona estrictamente

como una puerta de una sola vía hacia afuera. Le permite a tu servidor "salir" a Internet para descargar cosas (como cuando ejecutaste `apt update`), pero es un muro ciego desde el exterior.
Si alguien en otra casa, o incluso un familiar conectado a tu mismo router Wi-Fi físico, intenta escribir `10.0.2.15` en su navegador, su computadora arrojará un error inmediato. Esa dirección IP no existe en el Internet real, solo existe dentro de la mente de VirtualBox. Nadie de afuera puede usarla para entrar y ver tu página web.

La lógica de añadir el segundo adaptador (`enp0s8`)

Añadir una segunda tarjeta de red no encierra al servidor ni le quita libertades; es simplemente el acto de construir una segunda puerta en el mismo cuarto.

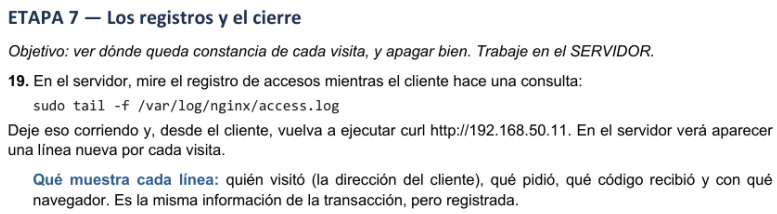
- **La Puerta 1 (NAT):** Sigue existiendo y funcionando. Sigue siendo la puerta trasera por donde tu servidor sale al mundo a comprar suministros e instalar programas.
- **La Puerta 2 (Red Interna):** Se construye como un pasillo cerrado y privado (tu red `tel223-lab`) que conecta exclusivamente la máquina Servidor con la máquina Cliente.

El sentido de usar esta arquitectura dual

El diseño de la guía tiene esta estructura porque busca darte lo mejor de dos mundos sin mezclar el tráfico:
Si solo tuvieras la red interna, tus dos máquinas virtuales podrían hablar entre sí perfectamente, pero sería imposible instalar Nginx porque ninguna tendría salida a Internet para descargar el catálogo de programas. Si solo tuvieras el adaptador NAT, tendrías Internet, pero como su diseño bloquea cualquier conexión entrante, VirtualBox te obligaría a realizar configuraciones de red muy avanzadas (llamadas "redirección de puertos") para lograr que el Cliente pudiera atravesar ese muro y ver la página del Servidor.
Al analizar cómo fluye el tráfico a través de estas tarjetas, tus dos sospechas iniciales resultan ser técnicamente incorrectas: tener solo NAT no expone tu página al mundo porque su dirección es falsa para el exterior, y añadir un segundo adaptador no restringe a la máquina,

sino que le da una nueva vía de comunicación exclusiva para tus experimentos.

ETAPA 7 - Los registros y el cierre



ETAPA 7 — Los registros y el cierre

Objetivo: ver dónde queda constancia de cada visita, y apagar bien. Trabaje en el SERVIDOR.

19. En el servidor, mire el registro de accesos mientras el cliente hace una consulta:

```
sudo tail -f /var/log/nginx/access.log
```

Deje eso corriendo y, desde el cliente, vuelva a ejecutar curl http://192.168.50.11. En el servidor verá aparecer una línea nueva por cada visita.

Qué muestra cada línea: quién visitó (la dirección del cliente), qué pidió, qué código recibió y con qué navegador. Es la misma información de la transacción, pero registrada.

▼ Lógica del comando sudo tail -f...



El comando `sudo tail -f /var/log/nginx/access.log` funciona como un monitor de seguridad en tiempo real para tu servidor.
Aquí tienes la lógica de cada componente paso a paso:

- `sudo` **(La llave maestra):** Como los registros guardan información sensible de la red (como las direcciones IP exactas de todos los que visitan tu página), Linux bloquea este archivo por defecto. Usas `sudo` para decirle al sistema: "Soy el administrador, tengo la autoridad para leer el diario de seguridad del equipo".
- `tail` **(Cola / Parte final):** En inglés, "tail" significa cola. Si tu servidor web lleva funcionando varios meses, su archivo de visitas puede tener millones de líneas de texto. Si intentas abrirlo con `nano`, tu computadora podría congelarse y tendrías que bajar infinitamente para ver las visitas de hoy. El programa `tail` está diseñado lógicamente para ignorar todo el pasado y leer únicamente las últimas 10 líneas del archivo.
- `f` **(Follow / Seguir):** Esta es la bandera que cambia por completo el comportamiento del comando. Sin la `f`, `tail` te mostraría las

últimas 10 visitas y se cerraría inmediatamente. Al agregar `f` (Follow), le ordenas a la computadora: *"No te cierres. Quédate 'enganchado' al final del archivo y síguelo vigilando. Cada vez que alguien escriba una línea nueva, imprímela inmediatamente en mi pantalla"*. Por eso, la instrucción te pide que dejes esto corriendo en el servidor mientras visitas la página desde tu cliente.

- `/var/log/nginx/access.log` **(La dirección del diario):** Es la ruta exacta hacia el archivo de texto que quieres vigilar.
  - `/var/` (Variable): Al igual que con `/var/www/`, el sistema operativo agrupa aquí todos los archivos que crecen, se achican o cambian constantemente de tamaño.
  - `log/` (Bitácora / Registro): Es el cuarto de archivos central. Aquí todos los programas de Linux tienen la obligación de guardar sus diarios de actividad para que el administrador pueda auditarlos.
  - `nginx/`: La carpeta exclusiva para los diarios de tu servidor web.
  - `access.log`: El nombre literal del archivo de texto. Es el "libro de visitas". Cada vez que `nginx` entrega una página web, tiene la orden interna de abrir este archivo y escribir una línea nueva detallando quién lo pidió, a qué hora, qué navegador usó y si la respuesta fue un éxito (200) o un error (404).

La lógica de usar este comando en el laboratorio es enseñarte cómo los ingenieros diagnostican redes en el mundo real. En lugar de adivinar si los mensajes del cliente están llegando al servidor, usas este comando para pararte metafóricamente detrás del hombro de Nginx y ver cómo anota cada visita en su libreta en el milisegundo exacto en que ocurre.

▼ Lógica de ejecutar curl sin filtros -I/-i

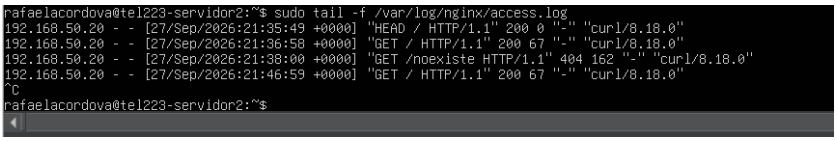


El programa `curl` (que significa "Client URL") es, en su forma más básica, un navegador web sin entorno gráfico. Dado que tu máquina virtual no tiene ventanas para abrir Google Chrome o Firefox, esta es la herramienta que usas en la terminal para comportarte como un visitante común que solicita información a otras computadoras en la red.
Como ya dominas el protocolo y el destino, el detalle técnico más importante de esta línea exacta es lo que **no** está escrito. A diferencia de los pasos anteriores de tu guía, aquí no estás usando ninguna bandera técnica (como la `-I` o la `-i`).
La lógica de ejecutar el comando completamente "desnudo" se basa en su comportamiento por defecto:

- **El enfoque en el usuario final:** Cuando usas `curl` sin letras adicionales, asume que no estás haciendo diagnósticos de red en ese momento. Va al servidor, recibe la respuesta completa, **tira a la basura automáticamente las cabeceras técnicas (el sobre) y escupe en tu pantalla únicamente el cuerpo del mensaje (el código HTML)**.
- **El sentido práctico:** Si lo piensas, esto es exactamente lo que hace tu teléfono móvil a diario. Cuando visitas un sitio web, tu navegador oculta los códigos `200 OK`, la fecha GMT y el nombre del servidor (Nginx) porque sabe que al humano solo le interesa el diseño visual. Al omitir las banderas, le estás dando a la terminal esa misma instrucción: *"Tráeme el archivo y muéstrame el contenido puro, no me aburras con la charla técnica entre las computadoras"*.

Es la forma más rápida de extraer el código fuente limpio de cualquier página web para guardarlo o leerlo.

```
rafaelacordova@tel223-cliente2:~$ curl http://192.168.50.11
<h1>Servidor TEL-223 - Puesto__</h1><p>Hola desde el servidor.</p>
rafaelacordova@tel223-cliente2:~$
rafaelacordova@tel223-cliente2:~$
rafaelacordova@tel223-cliente2:~$
```



▼ Lógica de los registros del archivo /var/log/nginx/access.log



El archivo `access.log` es el libro contable del servidor web. Leerlo es fundamental porque es la única herramienta que tiene un ingeniero para saber quién interactuó con el sistema, qué intentó hacer y cómo respondió la máquina en la capa invisible de la red.
Cada línea de este registro sigue un formato universal estricto. Aquí tienes la lógica forense de cada componente:

- `192.168.50.20` **:** Es la dirección IP del visitante. Como administrador, esto te dice exactamente desde qué máquina en la red se originó la solicitud.
- `-` **(Los primeros guiones):** En los registros de servidores, un guion () significa "campo vacío" o "dato inexistente". Estos dos espacios están reservados para protocolos antiguos de autenticación y contraseñas. Como tu página web es pública dentro de la red interna y no pide usuario ni contraseña para entrar, Nginx simplemente pone guiones.
- `[27/Sep/2026:21:35:49 +0000]` **:** La marca de tiempo exacta de la visita, incluyendo la zona horaria.

La Petición (Las primeras comillas)
Esto es exactamente lo que la máquina cliente le gritó al servidor al golpear el puerto 80.

- `HEAD` **o** `GET` **:** Son los "métodos" o verbos del protocolo HTTP.
  - Cuando ejecutaste `curl -I` (mayúscula), el programa usó el verbo `HEAD`, que lógicamente significa: *"Dame solo la cabecera, no quiero el cuerpo de la página"*.
  - Cuando ejecutaste `curl -i` o solo `curl`, el programa usó el verbo `GET`, que significa: *"Tráeme (Consigue) la página*

completa".

- `/` o `/noexiste`: Es la ruta del archivo que el cliente solicitó. La simple barra diagonal (`/`) significa "la página principal por defecto" (tu `index.html`).
- `HTTP/1.1`: El dialecto exacto de red que usó el cliente para hacer la pregunta.

El Resultado Técnico (`200 0`, `200 67`, `404 162`)
Estos dos números son la respuesta del servidor y el peso de lo que entregó.

- El `200` indica éxito (el archivo existía y se entregó), mientras que el `404` indica fracaso (el archivo no existe).
- El segundo número es el tamaño en bytes del cuerpo HTML que Nginx envió de vuelta por el cable.
    - ¿Por qué la primera línea dice `200 0`? Porque usaste `HEAD` (`curl -I`). El servidor fue exitoso (200), pero como le pediste estrictamente que no enviara el cuerpo de la página, el peso del documento entregado fue cero bytes.
    - En la segunda línea (`200 67`), pediste la página completa (`GET`), y el texto de tu archivo `index.html` pesaba exactamente 67 bytes.
    - En la tercera línea (`404 162`), pediste un archivo que no existía. Aunque falló (404), el servidor te envió de todas formas una página HTML de error por defecto, y esa página de error pesaba 162 bytes.

El Referer (`"-"`)
El penúltimo guion entre comillas indica "desde dónde llegó el usuario" (quién lo refirió). Si hubieras hecho clic en un enlace desde Google para llegar a tu servidor, aquí diría "www.google.com". Como ejecutaste el comando directamente en la terminal sin venir de otra página, el campo está vacío.

El Agente de Usuario (`"curl/8.18.0"`)
Esta es la "huella digital" del programa que hizo la visita.

- `curl` es el nombre del software, y `8.18.0` es la versión exacta de ese programa que está instalada en tu máquina Cliente. Si hubieras visitado la IP usando Firefox, aquí diría algo como `"Mozilla/5.0 Firefox/117.0"`.
- Aparece exactamente lo mismo en todas las peticiones porque las letras `I` o `i` solo son opciones temporales que le diste al programa para cambiar su método (`HEAD` o `GET`), pero la herramienta base que ejecutó la acción siempre fue el mismo programa `curl` versión 8.18.0.

Saber leer este registro es lo que diferencia a un usuario común de un administrador de sistemas. Si de repente ves cientos de peticiones con el método `GET`, buscando rutas extrañas como `/admin/contraseñas.txt`, y el Agente de Usuario dice `"Python-urllib"`, sabes instantáneamente, sin mirar ninguna otra pantalla, que no es un visitante humano, sino un robot automatizado intentando hackear tu servidor.

20. Salga del registro con Control C. Complete la evidencia de la página siguiente.

21. Apague las dos máquinas:

```
sudo poweroff
```

EVIDENCIA TÉCNICA DEL LABORATORIO 3

Esta es la evidencia de la sesión. La ficha pide capturas de la transacción: tome una captura de la salida de curl -i y péguela, o transcriba lo que vio.

La red

Pregunta	Mi respuesta
¿Cuántas máquinas virtuales levanté y qué rol tiene cada una?	En este laboratorio levanté una adicional a la que tenía, en total son 2 VM levantadas. Una es el servidor, la otra es el cliente.
¿Qué nombre de red interna usé en el Adaptador 2 de las dos?	tel223-lab
¿El ping del cliente al servidor funcionó? ¿Con cuánta pérdida?	Sí funcionó, con un 0% de pérdida.
¿Por qué la vez pasada no funcionaba y ahora sí?	No asistí la anterior vez.

La transacción HTTP

Consulta	Qué respondió el servidor (código y una cabecera)
Página que existe (curl -i http://192.168.50.11)	Código: 200 OK Cabecera: Content-Type: text/html
Página que no existe (curl -i .../noexiste)	Código: 404 Not Found, Cabecera: Server: nginx/1.28.3 (Ubuntu)

Pegue aquí su captura de la salida de curl -i, o transcríbala:

La captura de pantalla se encuentra en el pdf de evidencia del laboratorio.

Lo que aprendí hoy

Pregunta	Mi respuesta
¿Por qué una Red interna necesita que las máquinas estén en la misma computadora?	Porque, la Red Interna de Virtual Box está diseñada para solo conectar a VM que se encuentren en el mismo equipo físico.
¿Qué diferencia hay entre el código 200 y el 404?	200 es un código de la familia de éxito, 2xx, la solicitud del cliente ha sido completada con éxito 404, de la familia 4xx de error del cliente, indica que la URL está mal escrita
¿Para qué sirve el registro de accesos del servidor web?	Porque es la constancia de cada visita y transacción HTTP, incluyendo quién lo visitó (IP), cuando, qué solicitud ejecutó y el estado de retorno al cliente.

Anote lo que leyó en las tres consultas:

Consulta	Método	Código	Una cabecera que vio
curl -I (solo cabeceras)	HEAD	200 OK	Date: Sun, 27 Sep 2026 21:35:49 GMT
curl -i (página completa)	GET	200 OK	Server: nginx/1.28.3 (Ubuntu)
curl -i /noexiste	GET	404 Not Found	Content-Length: 162
PUNTO DE CONTROL — leyó una conversación HTTP real entre dos máquinas de su red. Esto es lo que un ingeniero mira cuando un servicio web no responde como debe.			